

UniversidadeVigo

ESCOLA SUPERIOR DE ENXEÑARÍA INFORMÁTICA

Memoria del Trabajo de Fin de Grado que presenta

Dña. Carlota Santiago Valverde

para la obtención del Título de Graduado en Ingeniería Informática

**Perfilado Estadístico y Orquestación Semántica para la Generación
Automatizada de Pipelines de Entrenamiento**



Junio, 2026

Trabajo de Fin de Grado N°: EI 25/26-125

Tutor: David Ruano Ordás

Cotutor: Jorge Rodríguez Cid

Área de conocimiento: Lenguajes y Sistemas Informáticos

Departamento: Informática

Índice de contenidos

1. Introducción.....	1
1.1 Objetivos.....	2
1.1.1. Objetivos principales.....	2
1.1.2. Objetivos específicos.....	3
1.2. Resumen de la solución propuesta.....	4
1.3. Metodología usada.....	5
2. Planificación y seguimiento.....	7
2.1. Planificación inicial.....	7
2.2. Seguimiento temporal.....	8
3. Arquitectura y tecnologías.....	10
3.1. Arquitectura del sistema.....	10
3.1.1. Módulos del Back-end.....	12
3.1.1.1. Módulo de clasificación de datos.....	12
3.1.1.2. Módulo orquestador.....	12
3.1.1.3. Módulo de entrenamiento de los modelos.....	12
3.1.2. Flujo de trabajo completo.....	13
3.2. Tecnologías e integración de productos de terceros.....	14
3.2.1. Entorno de entrenamiento (Sandbox).....	14
3.2.2. Front-End.....	14
3.2.3. Back-End.....	14
3.2.4. Herramientas de desarrollo.....	15
4. Especificación y análisis de requisitos.....	17
5. Diseño de software estático y dinámico.....	21
5.1 Arquitectura general del sistema.....	21
5.2 Estructura del Back-End.....	22
5.3 Diagrama de clases del sistema.....	23
5.3.1. API y modelos de Datos.....	23
5.3.2. Clase AnalysisFacade.....	24
5.3.3. Motor de Orquestación de Estrategias.....	25
5.3.4. Gestor de Modelos de Language (LLM).....	25
5.3.5. Proxy de Entrenamiento y Laboratorio.....	26
5.4 Diagramas de secuencia.....	26
5.4.1 Diagrama de secuencia general.....	26
5.4.2 Diagrama de secuencia de Análisis y Orquestación.....	27
5.4.3 Diagrama de secuencia del Laboratorio AutoML.....	27
6. Gestión de datos e información.....	29
6.1. Enfoque de gestión de privacidad.....	29
6.2. Datos almacenados por el sistema.....	29
6.3. Estructura de directorios.....	30
7. Aplicación a datos reales.....	31
7.1. Descubrimiento de Datos y Perfilado.....	31
7.1.1. Ingesta y Definición de Roles.....	31

7.1.2. Motor Estadístico y Análisis Técnico.....	31
7.1.3. Clasificación Semántica mediante LLM.....	33
7.2. Orquestación Autónoma y Entrenamiento.....	33
7.2.1. Evaluación de Dependencias y Estrategia.....	34
7.2.2. Generación de Código y Auditoría.....	34
7.2.3. Ejecución en Sandbox Virtualizado.....	34
7.3. Resultados del Entrenamiento AutoML.....	35
7.3.1. Rendimiento Global.....	35
7.3.2. Métricas detalladas por Objetivo.....	36
7.4. Conclusiones.....	37
8. Pruebas llevadas a cabo.....	37
9. Manual de usuario e instalación.....	39
9.1. Requisitos mínimos.....	39
9.2. Manual de instalación.....	39
9.2.1. Instalación en Windows.....	39
9.2.2. Instalación en Linux (Ubuntu 22.04).....	40
9.3 Manual de usuario.....	41
9.3.1. Carga del documento.....	41
9.3.2. Definición de Roles y Blindaje de Variables.....	42
9.3.3. Descripción de la Interfaz de Análisis y Orquestación.....	43
9.3.4. Interfaz de Auditoría de Código.....	44
9.3.5. Interfaz de Resultado de Entrenamiento.....	45
10. Conclusiones y trabajo futuro.....	47
10.1. Principales aportaciones.....	47
10.2. Conclusiones.....	47
10.3. Vías de trabajo futuro.....	48
11. Referencias.....	49
Anexo A. Diagramas de Gantt.....	51
Anexo B. Diagramas.....	53
Anexo C. Pruebas Realizadas.....	57

Índice de figuras

Figura 1. Comportamiento de la aplicación.....	4
Figura 2. Diagrama de Metodología Scrum.....	6
Figura 3. Diagrama de Metodología Scrum adaptada.....	7
Figura 4. Diagrama general de arquitectura del sistema.....	10
Figura 5. Diagrama de interacción entre módulos del Back-End.....	11
Figura 6. Visión general de los componentes del sistema.....	20
Figura 7. Estructura detallada del componente Back-End.....	21
Figura 8. Diagrama de la clase API y los modelos de datos.....	22
Figura 9. Diagrama de las clases de AnalysisFacade.....	23
Figura 10. Diagrama de clases de OrchestrationStrategy.....	24
Figura 11. Diagrama de clases de LLMClientStrategy.....	24
Figura 12. Diagrama de clases de TrainingServiceProxy.....	25
Figura 13. Manual de usuario - Selección de documento.....	40
Figura 14. Manual de usuario - Selección de roles y modelo LLM.....	41
Figura 15. Manual de usuario - Descripción Análisis y Orquestación.....	42
Figura 16. Manual de usuario - Visualización y Edición del Script.....	43
Figura 17. Visualización de las métricas finales.....	43
Figura 18. Diagrama de Gantt - Resumen de la Estimación.....	47
Figura 19. Diagrama de Gantt - Resumen de la Ejecución.....	48
Figura 20. Diagrama general de clases del sistema.....	49
Figura 21. Diagrama de secuencia general.....	50
Figura 22. Diagrama de secuencia de Análisis y Orquestación de Datos.....	51
Figura 23. Diagrama de secuencia de Laboratorio AutoML.....	52

Índice de tablas

Tabla 1. Planificación inicial.....	7
Tabla 2. Ejecución final.....	8
Tabla 3. Estructura teórica de una matriz de confusión binaria.....	34
Tabla 4. Resultados de la clasificación de especialidad (MSQ vs NO-MSQ).....	34
Tabla 5. Resultados de la clasificación de sala (H vs NH).....	35
Tabla 6. Resultados de la clasificación de prioridad (A, B o C).....	35
Tabla 7. Conjunto de pruebas de caja negra realizadas.....	54

Acrónimos y abreviaturas

CHUO: Complexo Hospitalario Universitario de Ourense

LLM: Large Language Models

HMR: Hot Module Replacement

OOM: Out Of Memory

RAG: Retrieval Augmented Generation

AutoML: Automated Machine Learning

MNAR: Missing Not At Random

API: Application Programming Interface

JSON: JavaScript Object Notation

CSV: Comma-Separated Values

ML: Machine Learning

HTTP: Hypertext Transfer Protocol

REST: Representational State Transfer

1. Introducción

Desde la irrupción de la informática en el ámbito científico a mediados del siglo XX, la capacidad de los sistemas computacionales para asistir en tareas de análisis de datos ha crecido exponencialmente. Las disciplinas que históricamente requerían años de formación especializada para extraer conocimiento a partir de grandes volúmenes de información han encontrado en el aprendizaje automático (Machine Learning, ML) un aliado de creciente relevancia. En las últimas décadas, la extensión de estas técnicas a los ámbitos empresariales, sanitarios e industriales ha generado una demanda sin precedentes de sistemas capaces de construir modelos predictivos de forma eficiente, reproducibles y adaptados a los datos disponibles [D1].

Sin embargo, pese a este avance tecnológico, la implementación del aprendizaje automático en entornos reales sigue presentando una brecha significativa entre su potencial teórico y su aplicabilidad operativa. Construir un pipeline de entrenamiento completo (desde la comprensión de los datos hasta la obtención de un modelo validado y listo para producción) sigue siendo, en gran medida, un proceso manual que exige decisiones técnicas en múltiples niveles de abstracción: (i) identificación de variables relevantes para el problema, (ii) determinación de la naturaleza estadística del problema, (iii) definición de estrategias de codificación y diseño, (iv) identificación de las dependencias existentes entre los objetivos de predicción, y (v) el reconocimiento y definición de las estrategias de modelado más apropiadas para la estructura concreta del conjunto de datos. Cada una de estas decisiones requiere tanto rigor estadístico como conocimiento del dominio, y un error en cualquier punto se propaga silenciosamente a lo largo de todo el proceso, afectando de forma irreversible a la calidad del modelo final [D2].

Con el objetivo de reducir esta carga, la comunidad científica ha desarrollado la automatización del aprendizaje automático (AutoML), que busca sistematizar la selección de algoritmos y la optimización de hiperparámetros. Los frameworks más consolidados en este ámbito (Auto-sklearn [D3], AutoGluon [D4] y H2O AutoML [D5]) reciben un conjunto de datos como entrada y devuelven directamente un modelo entrenado y serializado, sin exponer al usuario el proceso de construcción ni generar código inspeccionable. La única excepción parcial es TPOT [D6], que sí exporta un script Python ejecutable con el pipeline encontrado, si bien se limita a tareas de clasificación o regresión con un único objetivo y no incorpora ningún tipo de razonamiento sobre la naturaleza semántica de las variables ni sobre las relaciones estructurales entre los objetivos de predicción. En todos los casos, estos sistemas adolecen de tres limitaciones comunes: la imposibilidad de personalizar el pipeline según las restricciones específicas del dominio, la falta de transparencia en las decisiones tomadas internamente y la dependencia de servicios externos en la nube cuando los datos son sensibles [D7].

En paralelo a este desarrollo, la irrupción de los modelos de lenguaje a gran escala (Large Language Models, LLM) ha abierto nuevas posibilidades para automatizar tareas que hasta hace poco requerían intervención humana especializada. Trabajos recientes han demostrado que estos modelos presentan capacidades notables para la generación de código ejecutable y la comprensión del lenguaje natural, aplicadas a tareas de modelado predictivo [D8], y propuestas como *Text-to-ML* han explorado la síntesis completamente automatizada de pipelines de entrenamiento a partir de descripciones textuales de la tarea [D9]. La combinación de ambas tendencias (el análisis estadístico clásico como motor de caracterización y los LLM como orquestadores semánticos) abre la puerta a arquitecturas cualitativamente distintas de los enfoques tradicionales.

No obstante, esta convergencia no resuelve por sí sola los problemas estructurales que limitan la calidad de los pipelines generados automáticamente. El primero de ellos afecta al tratamiento de las variables categóricas. Los sistemas convencionales aplican codificaciones estándar sin distinguir si una variable es nominal (como el departamento al que pertenece un empleado, donde no existe ningún orden matemáticamente significativo) u ordinal (como su nivel jerárquico, donde la relación *Junior* < *Middle* < *Senior* es una estructura real que el modelo debería identificar, explotar y resolver

explícitamente). Como demuestran Zheng y Casari, utilizar codificación ordinal en variables nominales introduce relaciones artificiales entre categorías que pueden sesgar sistemáticamente el modelo, mientras que aplicar codificación *one-hot* en variables ordinales destruye la información de orden que contienen [D10].

El segundo problema estructural se presenta cuando deben predecirse simultáneamente varios objetivos (targets). Los sistemas convencionales tienden a tratarlas como tareas independientes, ignorando las relaciones de dependencia que puedan existir entre ellas. Esta estrategia, analizada en profundidad en la literatura sobre aprendizaje multiobjetivo, es estadísticamente correcta únicamente cuando los objetivos no están correlacionados entre sí [D11]. En presencia de dependencias, ignorar esta estructura implica desaprovechar información causal disponible en el propio conjunto de entrenamiento, lo que se traduce en modelos subóptimos que no captan la naturaleza conjunta del fenómeno que se pretende modelar.

Un tercer problema, de consecuencias aún más graves, surge cuando la ausencia de un valor en una columna objetivo no es aleatoria, sino que está estructuralmente determinada por el valor de otra variable. Este patrón se denomina, en la literatura estadística, *Missing Not At Random* (MNAR) [D12]. A diferencia de los mecanismos de ausencia aleatoria, el MNAR introduce un sesgo de selección que hace que los datos observados no sean representativos de la población en su conjunto. Un sistema que no detecta este patrón antes de iniciar el proceso de modelado entrena mezclando filas con datos reales y filas con ausencias estructurales, aprendiendo una correlación espuria que produce métricas de validación aparentemente aceptables, pero un modelo inútil en producción.

Con el objetivo de dar respuesta a esta problemática, el presente trabajo propone SPECTRA (Statistical profiling and semantic orchestration for the automated generation of training pipelines), una arquitectura que combina el análisis estadístico y el razonamiento semántico mediante modelos de lenguaje aplicados a conjuntos de datos para la creación automática de perfiles de entrenamiento y validación. El sistema se articula en tres etapas diferenciadas: (i) definición del problema, en la que el usuario establece los roles de las variables del conjunto de datos; (ii) perfilado estadístico, en la que se caracteriza exhaustivamente cada variable y se cuantifican las relaciones de información entre predictores y objetivos; y (iii) la orquestación de modelos, en la que un LLM interpreta el perfil estadístico obtenido, determina la estrategia de modelado más adecuada para la estructura concreta del problema y genera un script Python ejecutable, auditable y autorecuperable ante fallos de ejecución.

1.1 *Objetivos*

Los objetivos que se pretenden alcanzar en este sistema se dividen en dos objetivos principales y una serie de objetivos específicos, tal y como se detallan a continuación.

1.1.1. *Objetivos principales*

El objetivo principal de este proyecto consiste en el desarrollo e implementación de un framework de automatización y orquestación de pipelines de modelos de ML, capaz de seleccionar dinámicamente los mejores algoritmos de clasificación, estrategias de procesamiento y escenarios de validación según un conjunto de datos estructurados.

El segundo objetivo principal consiste en diseñar el entorno de ejecución aislado (*Sandbox*) gobernado por un bucle de autorreparación (Self-Healing Code) que garantice tanto la corrección de código como la estabilidad del pipeline de entrenamiento/validación generado, y que reduzca la intervención manual necesaria de personal técnico o de ingeniería de datos ante fallos de software rutinarios. Este entorno se implementará de forma controlada mediante contenedores de Docker.

1.1.2. Objetivos específicos

Para cumplir con los objetivos principales, se establecen los siguientes objetivos específicos según las fases del dato implementadas en el sistema:

OE1. Implementar el módulo de detección de orígenes de datos y protección de variables: desarrollar algoritmos encargados de verificar la integridad y la correcta lectura de los conjuntos de datos estructurados. Asimismo, se establecen mecanismos para asignar roles predictivos y blindar variables que el usuario considere críticas para la elaboración del sistema final. Esta característica asegura su tratamiento independientemente de su relevancia estadística.

OE2. Programar el motor local de perfilado estadístico y abstracción semántica: diseñar un componente encargado de clasificar los datos paramétricamente, emplear estrategias de imputación temporal y ajustar las distribuciones sesgadas con el fin de obtener un registro técnico en formato *TOON Dossier*, completando los subtipos de datos categóricos mediante modelos de lenguaje a gran escala (*LLM*).

OE3. Desarrollar el motor de orquestación para seleccionar la mejor arquitectura: construir un sistema de evaluación e inferencia estadística que analice las dependencias mutuas entre las variables objetivo (*targets*) para identificar automáticamente la arquitectura de entrenamiento más adecuada para abordar el modelado jerárquico y decidir dinámicamente si emplear técnicas de clasificación tradicionales o arquitecturas avanzadas adaptadas a escenarios con datos incompletos.

OE4. Automatizar la optimización de hiperparámetros y el blindaje contra la fuga de datos: diseñar e implementar heurísticas dinámicas que permitan seleccionar la mejor estrategia de búsqueda de hiperparámetros según el conjunto de datos. Este componente garantiza la separación entre los datos de entrenamiento y de validación para evitar sesgos predictivos.

OE5. Desarrollar el motor de generación automática de código de entrenamiento y validación: Diseñar e implementar un componente que traduzca las especificaciones de la arquitectura del *TOON Dossier* en un script ejecutable en *Python*, sin necesidad de conocimientos avanzados de programación.

OE6. Programar el bucle cerrado de detección y corrección de errores de codificación: Estructurar un sistema de resiliencia computacional (*Self-Healing Code Loop*) que capture en tiempo real las excepciones que lanza la consola, provocadas por los scripts generados, y clasifique los errores para guiar al *LLM*, en modo de depuración, a inyectar la corrección mínima de código sin la intervención humana.

OE7. Crear un entorno aislado (*Sandbox*) para la ejecución del software: desarrollar una infraestructura de virtualización ligera mediante contenedores a nivel del sistema operativo. Este entorno aislará la ejecución del script generado, garantizando así: (i) la seguridad del host, (ii) la replicabilidad del entorno de entrenamiento y (iii) la gestión de la persistencia o exportación de los modelos resultantes mediante archivos *.joblib*.

OE8. Evaluar experimentalmente el rendimiento del framework mediante un caso de estudio real y validar la eficacia de la solución propuesta. Se medirá la precisión de la orquestación de los modelos generados por el sistema.

Con el cumplimiento de estos objetivos se permitirá consolidar un sistema capaz de gestionar de forma autónoma el ciclo de vida de los datos, desde su carga y validación hasta la generación de la mejor configuración de modelos (orquestación de modelos) para el conjunto de datos indicado. Al final, se proporcionará un conjunto de métricas de evaluación que permitirán medir el rendimiento y la eficiencia de la solución generada.

1.2. Resumen de la solución propuesta

Para dar solución a los problemas planteados y cumplir con los objetivos genéricos establecidos, se ha desarrollado un sistema modular enfocado en la automatización del ciclo de vida del aprendizaje automático. La Figura 1 ilustra las tres etapas funcionales que conforman el flujo de trabajo agnóstico de la solución.

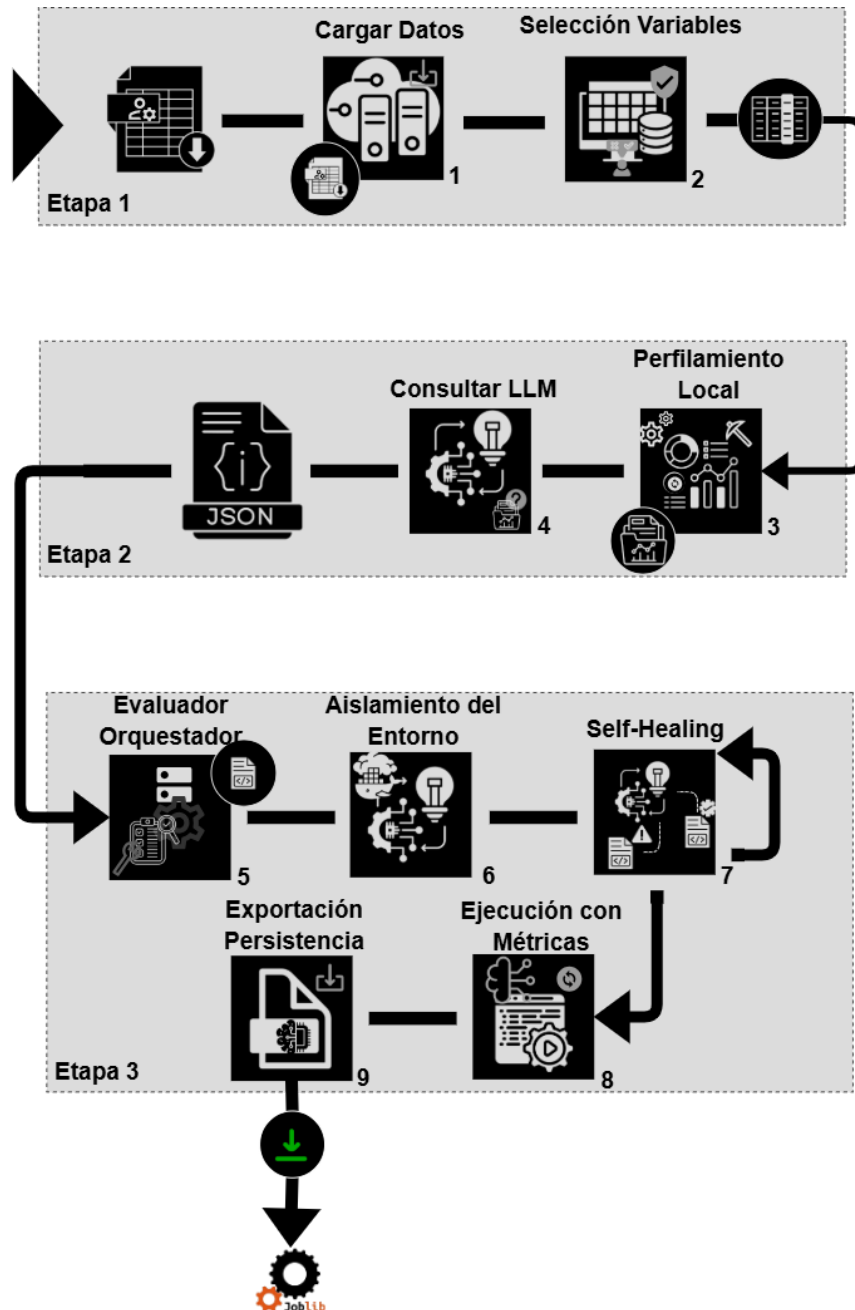


Figura 1. Comportamiento de la aplicación

Tal como se puede observar, la Etapa 1 se basa en la inserción de datos en la aplicación y se divide en dos fases fundamentales: (i) la carga de datos, en la que el sistema evalúa el archivo enviado y valida su extensión y formato; (ii) la selección de características, en la que se habilita una vista para indicar las características necesarias para la orquestación del sistema. Concretamente, la persona encargada decidirá unos roles (features para las variables que ayudan a predecir y targets para las variables

objetivo), además de establecer blindaje para que el LLM no descarte columnas por criterios puramente estadísticos.

A continuación, la Etapa 2 está compuesta por dos fases en las que se asume la responsabilidad del procesado y de la clasificación local de las variables. La primera fase (perfilamiento local) se encarga de determinar, mediante técnicas estadísticas, el tipo (binario, categórico, numérico y de alta cardinalidad) de cada una de las características previamente seleccionadas. A continuación, se realiza una consulta a un LLM para determinar si una variable categórica es ordinal (es decir, que existe un orden jerárquico) o nominal. Para ello, se construye un dossier con la información específica de las variables y una muestra de los datos, tal como se observa en la fase 4, etapa 2. El resultado de esta etapa genera un JSON que contiene los datos obtenidos tanto de forma local como mediante el LLM y sirve de enlace a la fase de resolución predictiva.

Por último, la Etapa 3 comprende tanto la orquestación automática del modelo final como el escenario de entrenamiento y validación, delegando en el sistema las decisiones de arquitectura, la generación automática de código y el despliegue del entorno seguro y autocorrectivo. Tal como se puede observar en la fase 5, etapa 3, se analizan los targets para determinar la arquitectura a implementar en función de sus dependencias y elegir el orden de orquestación. Estos datos se inyectan en el JSON de la etapa anterior para realizar otra consulta al LLM seleccionado por el usuario, que generará el script de ejecución para entrenar los modelos seleccionados. Como estos modelos de lenguaje tienden a fallar las primeras veces, se realiza una contenerización mediante Docker, como se observa en la fase 6, etapa 3, donde se ejecutará el script con métricas simples y un tiempo de espera más corto. En la fase 7 de la etapa 3 se visualiza la autocorrección del script si este contiene algún fallo, o Self-Healing, que intercepta los errores impresos en la consola e instruye al modelo en modo de depuración para corregir el script. Tras obtener un archivo funcional, en la fase 8 de la etapa 3 se muestra la ejecución del entrenamiento con métricas, utilizando los datos e hiperparámetros proporcionados por el usuario para crear los modelos finales, y se guarda en una carpeta local (como un archivo .joblib) para su reutilización en un despliegue técnico, como se muestra en la fase 9 de la etapa 3.

1.3. Metodología usada

Para garantizar la finalización y el cumplimiento de los plazos en la realización de este Trabajo de Fin de Grado (TFG), se ha optado por implementar un marco de trabajo basado en Scrum. Al tratarse de un entorno de investigación y desarrollo académico, donde prevalece un alto grado de incertidumbre, como la necesidad de asimilar nuevos conceptos técnicos a medida que el proyecto avanzaba, las metodologías de desarrollo tradicionales, basadas en enfoques predictivos, resultan ineficaces. En contraposición, las metodologías ágiles promueven un enfoque orientado a la flexibilidad y a la entrega continua de valor, facilitando la incorporación de cambios de manera iterativa en función de modificaciones tanto de ideas como de requisitos emergentes.

Scrum se basa en un marco de trabajo iterativo e incremental diseñado para organizar y gestionar proyectos complejos. A diferencia de los procesos estandarizados con pasos fijos, se basa en un conjunto de valores, principios y prácticas adaptables. El ciclo de vida del proyecto se gestiona mediante iteraciones de un periodo fijo denominadas Sprints, orientadas a mitigar los riesgos a largo plazo del proyecto mediante la obtención de incrementos funcionales del producto. Este proceso se gestiona mediante diversas herramientas visuales y eventos colaborativos. El núcleo inicial del proceso se centra en el Product Backlog, diseñado como un registro dinámico que recopila los requisitos, funcionalidades e historias de usuario y permanece abierto a modificaciones y sesiones de refinamiento continuas a medida que avanza el proyecto (Grooming). A partir de este bloque inicial y durante la planificación del Sprint, se recaba el Sprint Backlog, que agrupa los requisitos específicos que el equipo debe resolver a lo largo de esta etapa, desglosándose en tareas atómicas y manejables de baja complejidad.

Por otro lado, el seguimiento directo del estado de estas tareas se respalda en mecanismos de control visual, que constan de tableros de tareas como Scrum Boards y otros artefactos de difusión de información orientados a informar a todos los integrantes de forma clara y accesible. Finalmente, toda

la estructura se organiza y optimiza mediante los Eventos de Inspección y Adaptación, una agrupación de reuniones regulares que abarca la sincronización diaria o Daily Scrum, pasando por las revisiones del incremento funcional al cierre de cada ciclo, reconocidas como Sprint Review, hasta concluir con las sesiones de análisis retrospectivo o Sprint Retrospective, donde se evalúan las dinámicas del equipo para asegurar la mejora de todo el proceso de desarrollo.

Habitualmente, Scrum consta de tres roles esenciales con responsabilidades delimitadas: el Product Owner es responsable de mejorar el valor del producto y de gestionar el Product Backlog; el Scrum Master se encarga de guiar el proceso y de eliminar los impedimentos; y el Equipo de Desarrollo, compuesto por los profesionales encargados de construir el software. En la Figura 2 se muestra un diagrama del funcionamiento de Scrum.

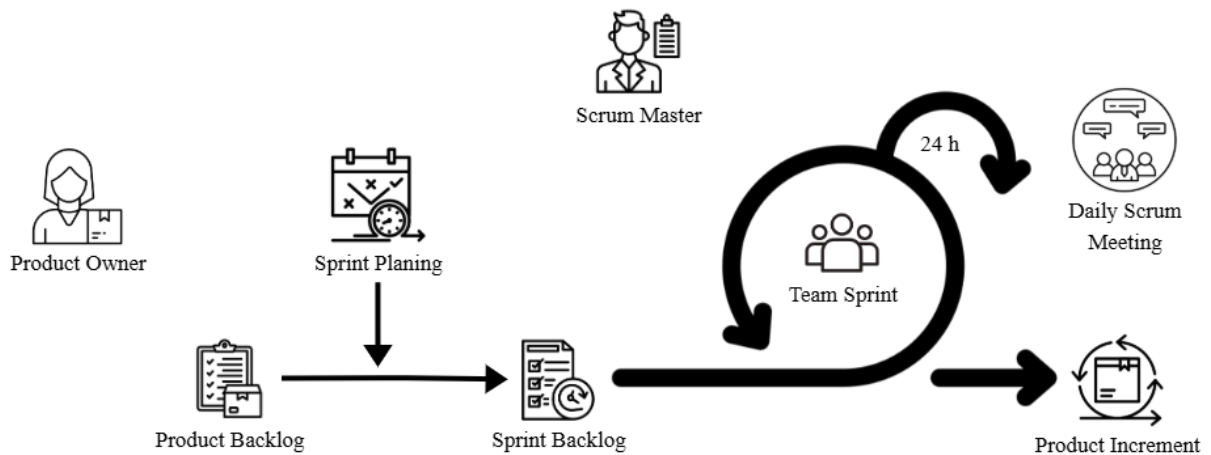


Figura 2. Diagrama de Metodología Scrum

Teniendo en cuenta las bases conceptuales sobre las que se asienta Scrum y la versatilidad que aporta al desarrollo del proyecto, se han ajustado algunos aspectos del marco original para adaptarlos a las pautas establecidas para el TFG. Una de las principales modificaciones se centra en el reparto de roles que establece Scrum. En este proyecto, el director David A. Ruano Ordás y el codirector Jorge Rodríguez Cid han asumido las responsabilidades de Scrum Master y Product Owner de forma conjunta y coordinada, encargándose de delimitar el alcance técnico, priorizar los objetivos de valor y actuar como proveedores del dominio del problema al abastecer los datasets con volantes médicos previamente corregidos y refinados por el actual codirector. A su vez, la alumna asume el rol principal del equipo de desarrollo por completo, siendo la única responsable de realizar tareas de análisis, diseñar la arquitectura, codificar, realizar pruebas e integrar la plataforma web.

Adicionalmente, los gráficos de Burn han sido reemplazados por un diagrama de Gantt [D13], una herramienta óptima para la visualización de actividades y tareas del proyecto a lo largo del cronograma global de la convocatoria, lo que proporciona una visión general y facilita el control de las desviaciones temporales respecto de la fecha límite de entrega. Por otro lado, la actividad del Scrum Diario se ha realizado mediante reuniones semanales de seguimiento, en las que se presentan los avances y se abordan las problemáticas o bloqueos sugeridos para ajustar la planificación de cara a la siguiente semana.

A continuación, en la Figura 3 se muestran todas las adaptaciones realizadas a la metodología Scrum tradicional.

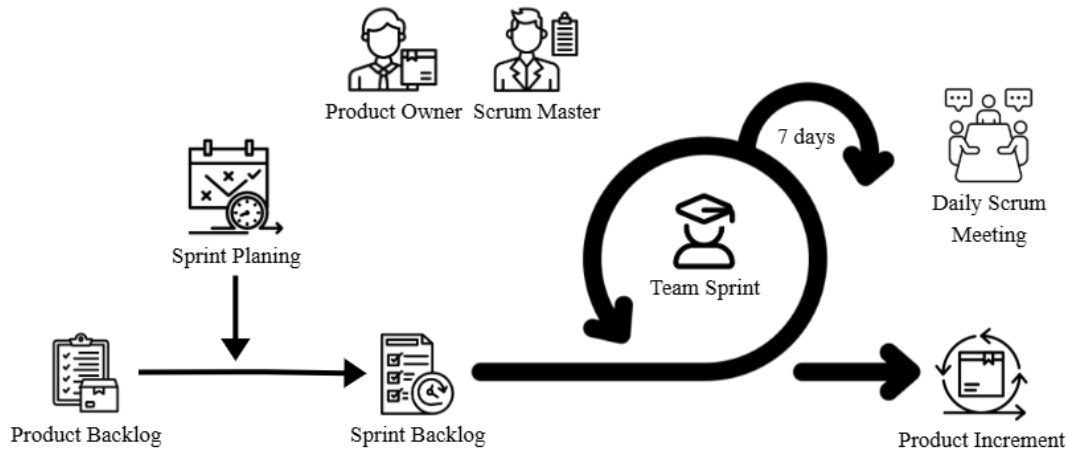


Figura 3. Diagrama de Metodología Scrum adaptada

Teniendo en cuenta las adaptaciones, se concluye que este enfoque se ajusta a las necesidades específicas de este Trabajo de Fin de Grado, disminuyendo los riesgos a largo plazo mediante iteraciones semanales en las que se agrega valor real al producto y se permite la asimilación de cambios y la incorporación de nuevos conceptos técnicos surgidos a lo largo del proceso de desarrollo.

2. Planificación y seguimiento

Esta sección muestra la comparación entre la planificación inicial de este trabajo y su ejecución real, mediante tablas y diagramas de Gantt.

2.1. Planificación inicial

Tras plantear las características principales y los objetivos del sistema SPECTRA, se estableció una planificación basada en la metodología Scrum adaptada, organizada en cuatro sprints de aproximadamente un mes de duración cada uno.

Dicho proyecto tuvo una organización temporal de 300 horas, distribuida entre el 19 de febrero y el 15 de junio de 2026. Este tipo de organización corresponde a un periodo de trabajo semanal de 17 u 18 horas, con un acumulado de 75 horas por sprint.

Antes de iniciar el desarrollo del sistema, se elaboró el Product Backlog inicial con tareas e hitos identificados para cubrir las tres etapas del sistema. La estimación se compuso en puntos de historia (PH), cada uno de los cuales equivale a 5 horas de trabajo, conforme a las prácticas recomendadas de la metodología Scrum.

En la Tabla 1 se muestra la planificación inicial, con las divisiones de Sprint, los hitos de desarrollo y los puntos de historia asignados a cada Sprint.

Tabla 1. Planificación inicial

Sprint	Fecha Inicio	Fecha Fin	Duración
Sprint 1	19 feb 26	19 mar 26	15PH (75h)
HU01 Carga, validación de integridad de datasets	19 feb 26	05 mar 26	9PH
HU02 Interfaz de selección de roles y declaración de variables blindadas	05 mar 26	19 mar 26	6PH

Sprint 2	19 mar 26	16 abr 26	15PH (75h)
HU03 Motor estadístico local	19 mar 26	02 abr 26	10PH
HU04 Clasificación semántica mediante LLM con cálculo dinámico	02 abr 26	16 abr 26	5PH
Sprint 3	16 abr 26	14 may 26	15PH (75h)
HU05 Evaluación orquestadora multiobjetivo de las dependencias de los targets.	16 abr 26	30 abr 26	7PH
HU06 Módulo de detección de ausencias estructurales y definición de la decisión sobre las estrategias.	30 abr 26	14 may 26	8PH
Sprint 4	14 may 26	15 jun 26	15PH (75h)
HU07 Generación automática del script de entrenamiento y prueba rápida de funcionamiento	14 may 26	22 may 26	2PH
HU08 Aislamiento del entorno de ejecución mediante <i>Sandbox</i>	22 may 26	29 may 26	2PH
HU09 Implementación del bucle cerrado de autocorrección del código	29 may 26	05 jun 26	2PH
HU10 Virtualización del código e interacción para entrenamiento pesado	05 jun 26	09 jun 26	5PH
HU11 Exportación de modelos serializados, preprocesados y logs	09 jun 26	15 jun 26	4PH
Total	19 feb 26	15 jun 26	60PH (300h)

En el Anexo A se presenta la representación de la estimación del proyecto mediante un diagrama de Gantt.

2.2. Seguimiento temporal

A medida que se iba empezando la ejecución real del proyecto, el tiempo de los sprints fluctuaba en comparación con lo que estaba estipulado, debido a contratiempos técnicos complejos que fueron surgiendo. Finalmente, el proyecto requirió de un total de 360 horas, superando las 300 horas establecidas con anterioridad, y la finalización de la implementación completa se modificó hasta el 22 de junio de 2026.

Tabla 2. Ejecución final

Sprint	Fecha Inicio	Fecha Fin	Duración
Sprint 1	19 feb 26	19 mar 26	75h
HU01 Carga, validación de integridad de datasets	19 feb 26	05 mar 26	45h
HU02 Interfaz de selección de roles y declaración de variables blindadas	05 mar 26	19 mar 26	30h

Sprint 2	19 mar 26	16 abr 26	80h
HU03 Motor estadístico.	19 mar 26	02 abr 26	50h
HU04 Clasificación semántica mediante LLM con cálculo dinámico	02 abr 26	16 abr 26	30h
Sprint 3	16 abr 26	14 may 26	85h
HU05 Evaluación orquestadora multiobjetivo de dependencias de targets	16 abr 26	30 abr 26	45h
HU06 Módulo de detección de ausencias estructurales y definición de decisión de estrategias	30 abr 26	14 may 26	40h
Sprint 4	14 may 26	22 jun 26	120h
HU07 Generación automática del script de entrenamiento y prueba rápida de funcionamiento	14 may 26	28 may 26	35h
HU08 Aislamiento del entorno de ejecución mediante <i>Sandbox</i>	28 may 26	04 jun 26	15h
HU09 Implementación del bucle cerrado de autocorrección del código	04 jun 26	15 jun 26	40h
HU10 Virtualización del código e interacción para entrenamiento pesado	15 jun 26	18 jun 26	15h
HU11 Exportación de modelos serializados, preprocesados y logs	18 jun 26	22 jun 26	15h
Total	19 feb 26	22 jun 26	360h

En el Anexo A se muestra el diagrama de Gantt que representa la ejecución real del proyecto.

La mayor desviación en el proyecto se concentró en el Sprint 4, con 120 horas reales frente a las 75 planificadas, debido a las complejidades surgidas en la automatización y en los mecanismos de autocorrección del código.

El primer gran obstáculo surgió en el HU07 (Generación automática del script de entrenamiento y prueba rápida de funcionamiento). Al delegar la escritura del código de entrenamiento al LLM, basándose en unas directrices semánticas de un dossier y en un prompt con las reglas y las acciones que debía realizar, el modelo tendía a cometer errores. Garantizar que el script autogenerado implemente un escenario de entrenamiento y validación completo y correcto demandó 35 horas de desarrollo, lo que incrementó los plazos inicialmente planificados.

El HU09 (Implementación del bucle cerrado de autocorrección del código) también retrasó la planificación inicialmente prevista debido a la imposibilidad de generar un script válido tras agotar los intentos de autorreparación del código o tras el timeout establecido por el usuario.

A pesar de estas desviaciones, el proyecto se finalizó con un retraso de solo una semana respecto a la planificación inicial, lo que permitió completar todas las funcionalidades previstas.

3. Arquitectura y tecnologías

A continuación se detallan los apartados relativos a las arquitecturas empleadas en el sistema para alcanzar los objetivos propuestos en la introducción.

3.1. Arquitectura del sistema

El sistema SPECTRA se ha diseñado desde un enfoque de arquitectura distribuida constituida por dos entornos de ejecución independientes, pero complementarios, como se muestra en la Figura 4:

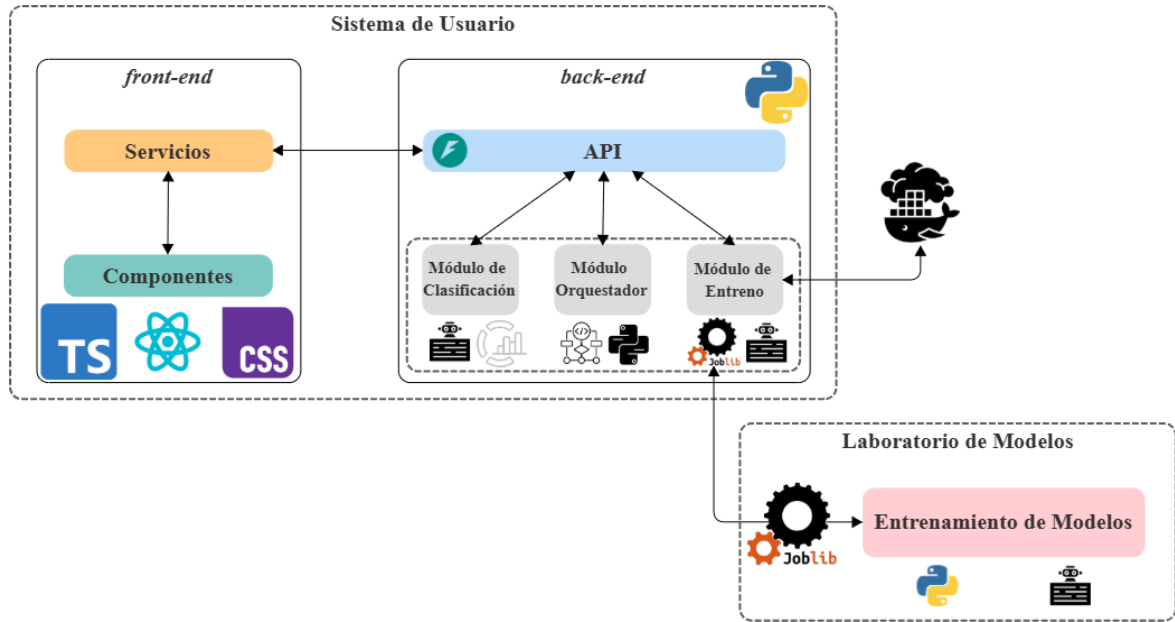


Figura 4. Diagrama general de arquitectura del sistema

Por un lado, se encuentra el Laboratorio de Modelos, un entorno experimental integrado para la orquestación y la validación automática del modelo final. En dicho componente se ejecutan (i) entrenamientos completos de aprendizaje automático basados en el script creado por el LLM, (ii) la optimización de los hiperparámetros proporcionados por el orquestador y (iii) la evaluación final de las métricas de rendimiento de cada entrenamiento. Los modelos entrenados y sus métricas se preservan mediante la biblioteca Joblib de Python para su uso posterior en la predicción.

Por otro lado, el Sistema de Usuario constituye el núcleo operativo que interactúa directamente con el usuario. El sistema sigue un enfoque de arquitectura modular distribuida mediante dos componentes principales que se ejecutan de manera local:

El *Front-End* gestiona la interfaz de usuario y la interacción con el usuario final. Está desarrollado como una Single Page Application (SPA) con React y TypeScript, lo que ofrece una experiencia fluida al cliente. La interfaz se distribuye en componentes modulares en los que se gestionan: (i) la carga y validación del documento de datos, (ii) la selección y clasificación de las variables, (iii) la gestión y selección de los parámetros de los modelos y (iv) la corrección y autogeneración del script de entrenamiento de modelos, así como la visualización de las métricas correspondientes.

El *Back-End* constituye el núcleo lógico y de orquestación del sistema, ya que es el encargado de coordinar toda la lógica, implementado mediante una API asíncrona con FastAPI. Este componente actúa como controlador central para dividirse en tres módulos especializados que se comunican de forma orquestada: (i) el Módulo de Clasificación utilizado para la ingesta y análisis semántico estadístico preliminar avalado en el LLM, (ii) el Módulo Orquestador es el encargado de decidir las

estrategias de la arquitectura mediante cálculos estadísticos y (iii) el Módulo de Entrenamiento se encarga de la ejecución y autocorrección del script de entrenamiento de modelos en un entorno de ejecución aislado (Docker) donde se crean y se validan.

La Figura 5 muestra el diagrama de flujo detallado del funcionamiento de la aplicación y de la interacción entre los distintos módulos del *Back-End*.

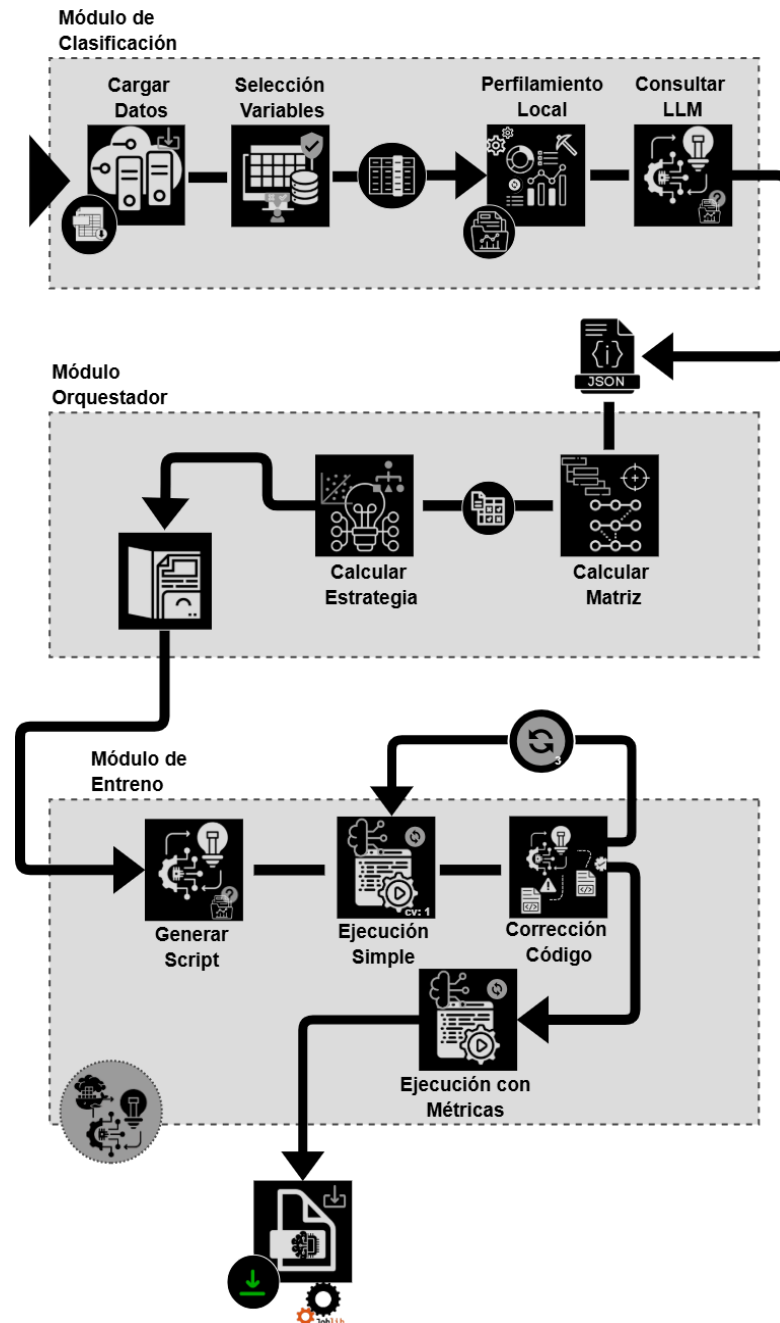


Figura 5. Diagrama de interacción entre módulos del Back-End

3.1.1. Módulos del Back-End

La organización establecida para el *Back-End* del proyecto constó de tres módulos, en los que se implementó la lógica principal para generar un script completamente funcional para el entrenamiento de modelos a partir de las variables seleccionadas por el usuario. A continuación, se detalla cada uno de dichos componentes, su funcionalidad y su integración en el flujo general del sistema.

3.1.1.1. *Módulo de clasificación de datos*

Este módulo, representado en la etapa 1 de la Figura 5, actúa como la entrada del sistema, recibe un dataset y verifica que el formato sea válido (CSV, Excel o Parquet). Esto garantiza que no exista ningún tipo de dato corrupto y que tenga una estructura correcta.

Una vez verificada la entrada, el servidor visualiza una tabla con los datos para realizar la selección de roles. En este paso se distinguen las variables que se utilizarán para predecir (features) y las que se predecirán (targets). Además, se permite seleccionar un blindaje de las variables críticas para mitigar el descarte automático del LLM por su escasa importancia matemática.

Tras seleccionar los roles, el motor interno de cómputo local realiza un análisis matemático exhaustivo del conjunto de datos. Sin variar el dataset, se calcula el nivel de cada variable junto con su subclase correspondiente. Si la columna es categórica, se realizará una consulta a un LLM local o remoto para determinar si existe un orden (si es ordinal) o no (si es nominal), enviando una muestra de cien filas con sus diferentes valores.

3.1.1.2. *Módulo orquestador*

Este componente constituye la organización estratégica del *Back-End*. El propósito principal es obtener una decisión de arquitectura lógica con hiperparámetros calculados estadísticamente a partir de los datos previamente recogidos en el JSON. Como se detalla en la etapa 2 de la Figura 5, este módulo ejecuta secuencialmente tres operaciones.

La primera de ellas es la **generación de la matriz de dependencias cruzadas** entre todos los targets, ya que muchos modelos tradicionales fallan por no tener en cuenta la existencia de variables que influyen en otras. Este componente analiza matemáticamente las relaciones de precedencia, correlación e impacto mutuo entre los targets declarados, lo que da como resultado un mapa topológico con las dependencias finales de cada par de variables, que determina el orden de ejecución del entrenamiento.

El **cálculo de la estrategia** se realiza a partir de la matriz del paso anterior. El módulo selecciona la arquitectura de aprendizaje automático óptima para realizar el entrenamiento multiobjetivo de manera completamente autónoma. Se evalúan las restricciones del dataset para decidir si se implementará una estructura dependiendo de la subclase de los targets: ClassifierChain si todas son categóricas, RegressorChain si son numéricas, HybridChain si son ambas, Multioutput si son independientes (la máxima dependencia no supera el umbral o threshold preestablecido) y GatedChain si existen ausencias estructurales.

Como tercera operación, se realiza la **inyección de las métricas** del usuario, en la que se recuperan e integran las directrices técnicas y las preferencias de evaluación establecidas por el usuario. El sistema acopla estas variables con las estrategias calculadas previamente y las envía, como manifiesto, al LLM seleccionado por el usuario para generar el script de entrenamiento.

3.1.1.3. *Módulo de entrenamiento de los modelos*

Este módulo, representado en la etapa 3 de la Figura 5, constituye la culminación del proceso y el componente de mayor complejidad. Su función es transformar las decisiones de diseño arquitectónico y las métricas seleccionadas en un script de modelos de ML funcional mediante un proceso automatizado de codificación, corrección y optimización. El módulo consta de tres fases críticas.

Generación Automatizada del Script. El módulo recoge el JSON de la anterior etapa para utilizarlo como contexto dentro de un prompt técnico que se envía al LLM junto con unas métricas del sistema para que el modelo sea capaz de generar un script completo del entrenamiento partiendo de los targets.

Fase de Corrección de Código. Dado que los modelos LLM pueden introducir varios fallos de síntesis, el sistema ejecuta el script generado en un entorno controlado. Este despliega el script dentro de un contenedor Docker, donde el *Back-End* parsea las métricas finales definidas por el usuario, utilizando una configuración mínima para que la ejecución de la corrección no sea demasiado lenta. Si el script contiene algún fallo, el contenedor recoge todos los errores de la consola para realizar un saneamiento de código mediante un LLM, con un prompt seleccionado para eliminar sólo los errores y volver a ejecutarlo dentro del contenedor para validarlo. Este proceso se repetirá hasta que el LLM sea capaz de codificar un script completamente funcional libre de excepciones.

Fase de Entrenamiento Pesado. Cuando el script generado sea correcto y se ejecute por completo, dicho script se muestra al usuario. Tras la validación o confirmación de los datos y las métricas, el sistema vuelve a lanzar el contenedor Docker para realizar el entrenamiento y mostrar al final de su ejecución la estadística de cada modelo implementado para cada target.

3.1.2. Flujo de trabajo completo

El proceso completo, como se ilustra en la Figura 5, comienza con la carga de un conjunto de datos (en formato csv, Excel o Parquet) a través de la interfaz de usuario. Este documento se envía al *Back-End*, donde se valida el formato y se extrae el dataset para su visualización.

A continuación, el usuario selecciona los roles de cada variable (target, features o features blindadas) para aplicar el módulo de clasificación a ellas. Durante este proceso, se realiza una clasificación detallada mediante procesos internos de *Back-End* y llamadas a un modelo LLM seleccionado por el usuario anteriormente. Esto da como resultado un archivo JSON con toda esta clasificación detallada como su tipo y subtipo o las information gain scores de cada feature con cada target.

Estos datos se envían al módulo de orquestación para seleccionar la mejor estrategia de modelado para el conjunto de targets, utilizando una matriz de dependencia cruzada para una mejor elección.

Tras la selección de la estrategia, se le presenta al usuario el JSON con los datos del primer y del segundo módulo, donde podrá realizar correcciones si el modelo LLM dio un valor erróneo y escoger las métricas del entrenamiento de los targets.

Todos estos datos se procesan e inyectan en un prompt para enviarlo a otro modelo LLM previamente seleccionado y así generar el script de entrenamiento. Una vez finalizada la respuesta, dicho script se ejecutará en un entorno controlado para mayor seguridad. Si el script contiene errores de cualquier tipo, el contenedor Docker envía los errores a la consola para volver a lanzar una pregunta al modelo para que corrija las líneas incorrectas de código. Este paso se realizará un máximo de tres veces para hallar la corrección del archivo.

Una vez finalizada la corrección, el sistema muestra al usuario el script para verificar que las métricas que seleccionó con anterioridad consten en él. Tras la confirmación del usuario y la ejecución del script en Docker, los archivos de entrenamiento con formato .joblib y las métricas globales y específicas de cada modelo se guardarán en el *Back-End* para su posterior utilización y se mostrarán al usuario las especificaciones de cada modelo con su porcentaje de acierto tanto global como individual.

3.2. Tecnologías e integración de productos de terceros

En este apartado se describen las diversas herramientas y tecnologías empleadas en la implementación del proyecto.

3.2.1. Entorno de entrenamiento (Sandbox)

El entorno de modelos constituye un espacio de ejecución aislada donde se valida, depura y ejecuta el script generado por el LLM para el entrenamiento de aprendizaje automático. A diferencia de un entorno experimental manual, este sistema está completamente automatizado para generar un archivo funcional que permita entrenar los mejores modelos para cualquier variable de un dataset. Para aislar la ejecución se pueden dos enfoques.

Docker: se utiliza como plataforma principal para proporcionar un entorno de contenedores aislado y seguro. Si está disponible, el script se ejecuta sin acceso al sistema, lo que garantiza el aislamiento y la seguridad del sistema local.

Subprocesos: es el sistema de respaldo en caso de no disponer de Docker; utiliza un módulo de subprocesos de Python para levantar un entorno de ejecución ligero.

3.2.2. Front-End

El *Front-End* constituye la capa de visualización del sistema, siendo responsable de proporcionar una interfaz intuitiva y eficiente para que los usuarios interactúen con SPECTRA, definan los datos y visualicen las métricas de entrenamiento.

Vite: entorno de desarrollo para aplicaciones web modernas que ofrece un servidor de Hot Module Replacement (HMR) instantáneo y un sistema de compilación optimizado.

React: biblioteca de JavaScript de código abierto diseñada para construir interfaces de usuario declarativas y basadas en componentes modulares.

TypeScript: superconjunto tipado de JavaScript que añade tipos estáticos opcionales al lenguaje, proporcionando un código robusto, con menos errores y de fácil escalabilidad.

Zustand: solución de estado global minimalista, rápida y escalable. Facilita el flujo de datos complejo entre componentes.

Lucide React: biblioteca de iconos vectoriales limpios y de calidad para sustituir soluciones más pesadas, integrándose en el diseño moderno de la interfaz.

3.2.3. Back-End

El *Back-End* constituye el núcleo funcional del sistema, donde se implementan los módulos de clasificación, orquestación y entrenamiento del dataset. Esta sección describe las tecnologías y bibliotecas especializadas que se han integrado en su construcción.

Python: es un lenguaje de programación interpretado, de alto nivel y propósito general, conocido por su legibilidad y simplicidad sintáctica. Es uno de los lenguajes más conocidos para el análisis de datos, el aprendizaje automático y el desarrollo web, gracias a las librerías y frameworks que se han desarrollado. Es un lenguaje cuya sintaxis permite expresar conceptos con menos líneas de código y prioriza la legibilidad.

FastAPI: es un framework web de alto rendimiento para la construcción de APIs con Python, basándose en la validación de tipos estándares (Pydantic). Ha sustituido frameworks tradicionales como *Flask* por su soporte para la asincronía y su alto rendimiento.

Scikit-learn: es la biblioteca de aprendizaje automático fundamental en Python para realizar el análisis y el modelado de datos subyacentes. En el proyecto se implementan tanto el cálculo del *Information Gain* como los scripts de entrenamiento y los pipelines predictivos.

Pandas: biblioteca estándar para el análisis y la manipulación de datos. Proporciona estructuras de datos flexibles y herramientas para trabajar con datos estructurados o con series temporales. La estructura principal de esta biblioteca es el *DataFrame*, una tabla bidimensional que facilita la manipulación, el filtrado y el análisis de grandes conjuntos de datos. En el sistema se utiliza para el cálculo de distribuciones, la creación de matrices de confusión y el muestreo estratificado preciso.

Joblib: biblioteca de Python que proporciona herramientas para la paralelización ligera y la persistencia de datos. Se ha utilizado para serializar los modelos entrenados y guardarlos en disco para su despliegue posterior en producción.

Ollama (API): actúa como servidor local o remoto para ejecutar LLMs. Se ha integrado para manejar tanto las consultas de clasificación de datos (clasificación ordinal o nominal) como la generación y reparación del script de entrenamiento de código del Orquestador.

Optuna: se trata de un framework software de código abierto para optimizar los hiperparámetros automáticamente. En el sistema, el LLM inyecta Optuna en el script generado para realizar una búsqueda bayesiana inteligente, siendo la estrategia elegida por defecto al ejecutar pipelines complejos o arquitecturas de múltiples targets.

3.2.4. Herramientas de desarrollo

Un sistema complejo como el presentado en este Trabajo de Fin de Grado (TFG) requiere unas herramientas especializadas para facilitar la colaboración, controlar las versiones, visualizar los datos y la documentación del proyecto. En esta sección se describe el conjunto de herramientas utilizadas para dar soporte al proceso de desarrollo, asegurando así una gestión eficiente y una implementación de calidad.

Visual Studio Code: entorno de desarrollo ampliamente utilizado de código multiplataforma compatible con Windows, Linux y macOS, desarrollado por Microsoft. Ofrece herramientas avanzadas como la depuración nativa, la integración directa con repositorios de Git, el autocompletado semántico, el resaltado de sintaxis y las opciones de refactorización. Se trata de la plataforma más usada gracias a su alto grado de personalización, que otorga al desarrollador total libertad para modificar atajos de teclado, temas visuales y ajustes del entorno de trabajo.

Git: sistema de control de versiones de código abierto, robusto y distribuido, diseñado para la gestión de proyectos de software de cualquier magnitud. Es una herramienta fundamental para el trabajo en equipo, ya que permite la modificación simultánea del código por varios desarrolladores, coordina el flujo de trabajo y mantiene un historial detallado de las modificaciones sin generar conflictos.

GitHub: plataforma web de alojamiento y colaboración más extendida en la industria del desarrollo de software, basada en la tecnología de control de versiones de Git. Esta ofrece a los desarrolladores un espacio en la nube para almacenar el código de sus proyectos, facilitando además los proyectos colaborativos. Es posible rastrear el historial de modificaciones, revisar las contribuciones de los diferentes autores y sincronizar el proceso de desarrollo de una forma transparente.

npm: administrador de dependencias estándar del ecosistema de Node.js, el entorno principal para ejecutar JavaScript. Se compone de una interfaz de línea de comandos conectada a un registro global extenso de bibliotecas de código abierto. El propósito de esta herramienta es facilitar la instalación de módulos de terceros, la publicación de sus desarrollos propios y el mantenimiento de las dependencias de la aplicación.

Cacao: plataforma web orientada al diseño de diagramas técnicos con un acabado profesional. Gracias a su interfaz amigable, agiliza la representación visual de los procesos. En este Trabajo de Fin de Grado se han utilizado diagramas de arquitectura y esquemas explicativos para su elaboración.

ProGantt: plataforma web para diseñar y administrar diagramas de Gantt. Se enfoca en la organización temporal, en la que se definen las tareas, los hitos clave y las dependencias a lo largo de un cronograma. Este recurso es vital para la optimización de la productividad y para llevar un control del tiempo de cada subapartado.

4. Especificación y análisis de requisitos

Esta sección enumera las historias de usuario que forman parte del Product Backlog siguiendo una misma estructura:

- Nombre breve y descriptivo
- Descripción de la funcionalidad
- Criterio de validación y verificación que determina cuándo se considera terminada y aceptable el desarrollo de la funcionalidad

HU01 Carga, validación de integridad de datasets

Como usuario administrativo

Quiero cargar el archivo de datos con diferentes formatos (CSV, Excel y Parquet)

Para validar la integridad de los datos antes del inicio de procesamiento de modelado

Criterios de aceptación:

- El sistema acepta archivos en los formatos .csv, .xlsx y .parquet.
- La interfaz debe proporcionar opciones para arrastrar y soltar documentos o seleccionarlos mediante el explorador de archivos
- El sistema debe extraer correctamente el texto del archivo de datos, independientemente del formato.

HU02 Interfaz de selección de roles y declaración de variables blindadas

Como usuario administrativo

Quiero disponer de una vista interactiva para asignar los roles a las variables del dataset

Para definir cuáles son las columnas que actuarán como características (features), cuáles como objetivos (target) y cuáles deben ser ignoradas

Criterios de aceptación:

- La interfaz debe listar todas las columnas detectadas en el archivo cargado.
- Se debe permitir que el usuario decida asignar el rol específico a cada variable haciendo clic en la columna correspondiente.
- La interfaz debe permitir al usuario seleccionar o deseleccionar columnas mediante un botón desplegable.
- La interfaz debe poder dejar filtrar la información de cada columna al usuario.
- El sistema debe validar obligatoriamente que se haya seleccionado al menos una variable de entrada y una variable objetivo antes de permitir avanzar.

HU03 Motor estadístico local

Como usuario

Quiero clasificar mediante descriptores matemáticos el tipo de cada columna seleccionada para hallar las subclases menos las categóricas

Para extraer la clase y subclase para definir los tipos de datos

Criterios de aceptación:

- El *Back-End* debe ejecutar automáticamente el motor estadístico al aceptar la selección de roles en el HU02.
- El motor estadístico debe calcular la clase, la subclase, el ig-score y la entropía por pares.
- Los cálculos realizados deben estructurarse y guardarse en un archivo JSON para servir de base al módulo de inferencia.

HU04 Clasificación semántica mediante LLM con cálculo dinámico

Como usuario administrativo

Quiero lanzar un dossier con los valores estructurados y clasificados de las variables para hallar las subclases de las columnas categóricas

Para estructurar los datos categóricos mediante ordinales y nominales

Criterios de aceptación:

- El sistema debe enviar un dossier con los metadatos y los prompts a un modelo de LLM alojado localmente o de forma remota en el servidor de Ollama.
- La identificación de las subclases debe realizarse de forma asíncrona para garantizar la privacidad de los datos.
- La respuesta del LLM debe inyectarse en el dossier enviado para guardarlo localmente.

HU05 Evaluación orquestadora multiobjetivo de dependencias de targets

Como usuario administrativo

Quiero que el sistema analice las dependencias cruzadas entre los targets seleccionadas

Para descubrir las relaciones estadísticas entre los targets

Criterios de aceptación:

- El orquestador debe evaluar la asociación matemática entre los pares de targets mediante métricas como el coeficiente de correlación de Spearman, el de Kendall o la prueba de Kruskal-Wallis.
- El sistema debe ser capaz de procesar escenarios multiobjetivos de forma automática.
- Los coeficientes obtenidos deben visualizarse para determinar la estrategia óptima.

HU06 Módulo de detección de ausencias estructurales y definición de decisión de estrategias

Como usuario administrativo

Quiero detectar el mecanismo de los datos ausentes y determinar la arquitectura de modelado idónea

Para mitigar los sesgos matemáticos aplicando estrategias avanzadas como GatedChain o ClassifierChain de modelado idónea

- Criterios de aceptación:
- El sistema debe evaluar si el patrón de nulos se debe a ausencias estructurales graves (MNAR).
- En función de los nulos y de la dependencia calculada para cada target en HU06, el orquestador elige la plantilla de código más robusta.
- La estrategia debe guardarse en los metadatos de la sesión.

HU07 Generación automática del script de entrenamiento y prueba rápida de funcionamiento

Como usuario administrativo

Quiero que el sistema genere el script base y lo ejecute bajo una configuración mínima para ver si no hay errores en compilación

Para comprobar la viabilidad del código sin comprometer tiempo de cómputo elevado

Criterios de aceptación:

- El sistema debe generar un script automático, mediante un LLM, que incluya todo el pipeline de preprocesamiento y entrenamiento de los modelos.
- El *Back-End* para inyectar el parámetro de validación rápida que desactiva el entrenamiento pesado.
- El script se enviará automáticamente al entorno de ejecución aislado para su validación inicial.

HU08 Aislamiento del entorno de ejecución mediante *Sandbox*

Como usuario administrativo

Quiero aislar la ejecución de los scripts dentro de un entorno virtualizado seguro

Para evitar que los códigos defectuosos afecten a la estabilidad, seguridad o archivos del sistema local

Criterios de aceptación:

- El script debe ejecutarse dentro del contenedor aislado para la prevención de seguridad.
- El entorno debe capturar los errores de la consola para realizar la autocorrección.

HU09 Implementación del bucle cerrado de autocorrección del código

Como usuario administrativo

Quiero que el sistema intercepte los errores del contenedor los envíe de vuelta para su autorreparación

Para solucionar fallos de forma autónoma sin requerir una intervención manual

Criterios de aceptación:

- El sistema captura los errores de compilación del script que se muestran en la consola.
- El módulo de corrección debe enviar al LLM el mensaje de fallo para solicitarle un parche correctivo.
- El proceso se repite de forma interactiva hasta que el script se ejecute sin errores o que se repita 3 veces.

HU10 Virtualización del código e interacción para entrenamiento pesado

Como usuario administrativo

Quiero visualizar el script definitivo ya verificado con el valor de confianza del modelo y seleccionar las métricas de alta carga por el usuario

Para configurar manualmente los parámetros reales de cross-validation y la optimización de hiperparámetros

Criterios de aceptación:

- El sistema mostrará el script corregido que superó la prueba de ejecución rápida.
- La interfaz debe proporcionar los controles para que el usuario seleccione el número de cv-folds definitivo y la estrategia de optimización.
- El sistema debe recopilar la configuración del usuario e inyectarla en el script para ejecutar el entrenamiento real de los modelos.

HU11 Exportación de modelos serializados, preprocesados y logs

Como usuario

Quiero que el sistema complete el entrenamiento de alta carga y exporte los artefactos optimizados

Para descargar y reutilizar los modelos predictivos y sus respectivos pipelines en entornos de producción

Criterios de aceptación:

- Los modelos resultantes deben serializarse en un Laboratorio de Modelos en formato .joblib.
- La interfaz debe habilitar un panel de resultados que muestre las métricas reales de rendimiento y un mensaje que indique que el entrenamiento se ha completado.

5. Diseño de software estático y dinámico

En esta sección se mostrarán los elementos del diseño de software más importantes desarrollados en este proyecto, tanto desde una perspectiva estática (componentes y clases) como desde una dinámica (secuencia).

5.1 Arquitectura general del sistema

En la Figura 6 se presenta una visión general de los componentes principales del sistema. Como se puede observar, está formada por tres grandes bloques: (i) *Front-End*, (ii) *Back-End*, y (iii) Laboratorio de Modelos.

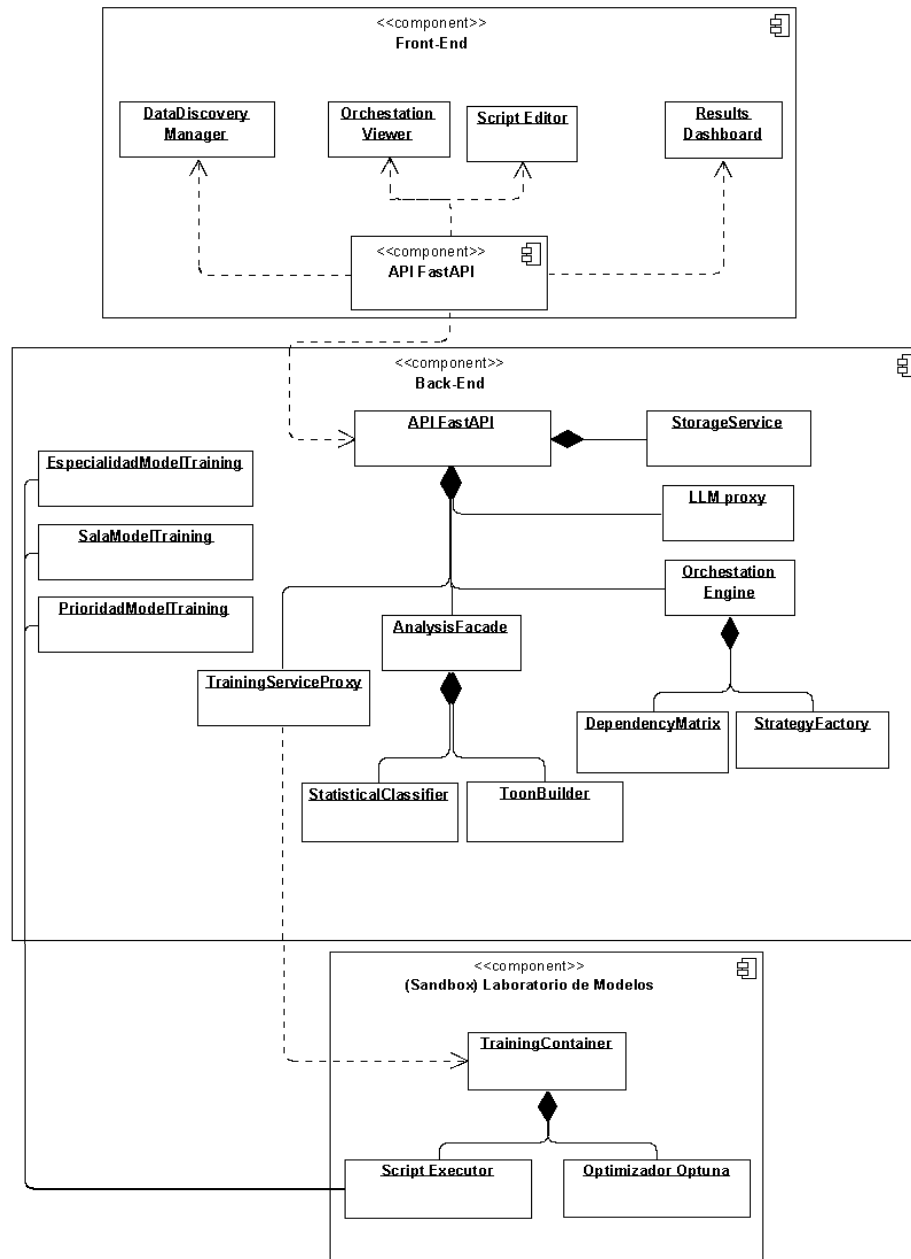


Figura 6. Visión general de los componentes del sistema

El *Front-End* contiene los componentes de la interfaz del usuario, lo que permite la interacción del usuario con el sistema. Está compuesto por cinco componentes principales. Todos estos componentes se comunican con la aplicación a través de la web central. Por otro lado, el *Back-End* constituye el núcleo del sistema y alberga la API, sirviendo de puente entre la interfaz del usuario y la lógica de negocio. Este componente gestiona la clasificación estadística de las variables seleccionadas, la selección de estrategias, la generación del script y el entrenamiento de los modelos.

Finalmente, el *Laboratorio de modelos* contiene los componentes dedicados al entrenamiento de los diferentes modelos de clasificación utilizados por el sistema de usuario

5.2 Estructura del Back-End

Para una mejor visión y comprensión del sistema, se ha elaborado un diagrama detallado del componente *Back-End*, como se observa en la Figura 7, ya que es el componente más importante de este trabajo.

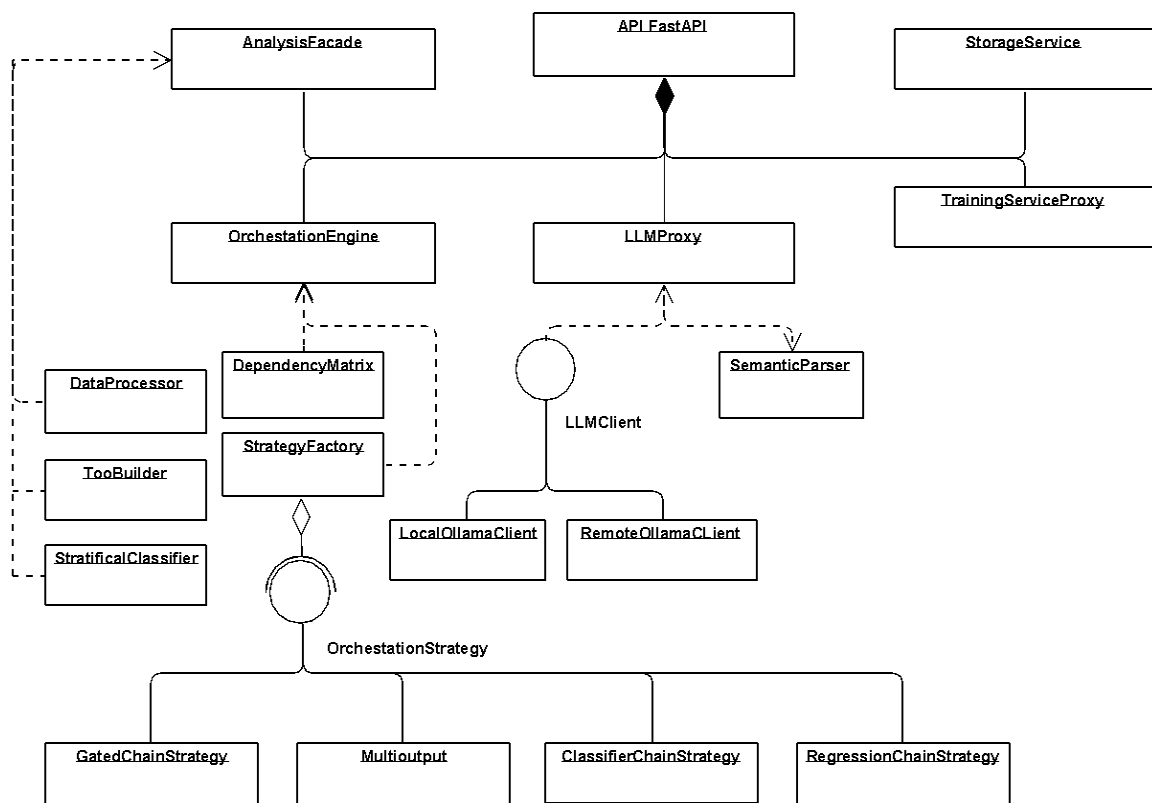


Figura 7. Estructura detallada del componente Back-End

Este módulo tiene como componente central la API (FastAPI), que actúa como la entrada de datos provenientes del *Front-End* y orquesta el flujo de trabajo hacia los demás componentes. Esta API está conectada con cinco componentes principales:

AnalysisFacade. Encargado de centralizar el descubrimiento de datos y el perfilado estadístico de las variables.

OrchestrationEngine. Responsable de crear la matriz de dependencias y de determinar la estrategia óptima de entrenamiento.

LLMProxy. gestiona la comunicación entre modelos de LLM, tanto en local como en remoto.

TrainingServiceProxy. Enviando la generación del script y el entrenamiento de los modelos al entorno aislado del *Sandbox*.

StorageService. Centraliza la lectura dinámica de los conjuntos de datos de los usuarios y la escritura de los modelos entrenados resultantes.

Estos componentes, a su vez, se apoyan en distintos elementos del sistema. En concreto, el **AnalysisFacade** gestiona el cálculo matemático del **StatisticalClassifier**, la limpieza de valores nulos del **DataProcessor** y la empaquetación final del metadato mediante el **ToonBuilder**. Por otra parte, el **OrchestrationEngine** evalúa y genera la matriz de dependencia entre las variables mediante el **DependencyMatrix** y utiliza un patrón fábrica (**StrategyFactory**); dicha fábrica aplica un patrón estrategia a través de la interfaz **OrchestrationStrategy** para instanciar la estrategia correspondiente a cada conjunto de datos, como **GatedChainStrategy**, o algoritmos genéricos como **MultiOutput**. Finalmente, el **LLMProxy** parsea las respuestas de la IA mediante el **SemanticParser** y asegura la escalabilidad del sistema implementando la interfaz **LLMClient**, lo que permite conectarse a motores locales (**LocalOllamaClient**) o externos (**RemoteOllamaClient**) sin alterar el núcleo lógico de la aplicación.

5.3 Diagrama de clases del sistema

En la Figura 20, añadida al Anexo B para una mejor visualización, se muestra el diagrama completo de clases del subsistema *Back-End* de SPECTRA, que, dada su complejidad, se ha dividido en varios diagramas específicos para facilitar su comprensión.

5.3.1. API y modelos de Datos

La clase API, visualizada en la Figura 8, actúa como el controlador del sistema entre el *Front-End* y los demás componentes del sistema *Back-End*. En este apartado se muestran tanto los métodos y atributos del componente como los modelos utilizados para el envío de datos y la creación de la interfaz de usuario mediante peticiones HTTP.

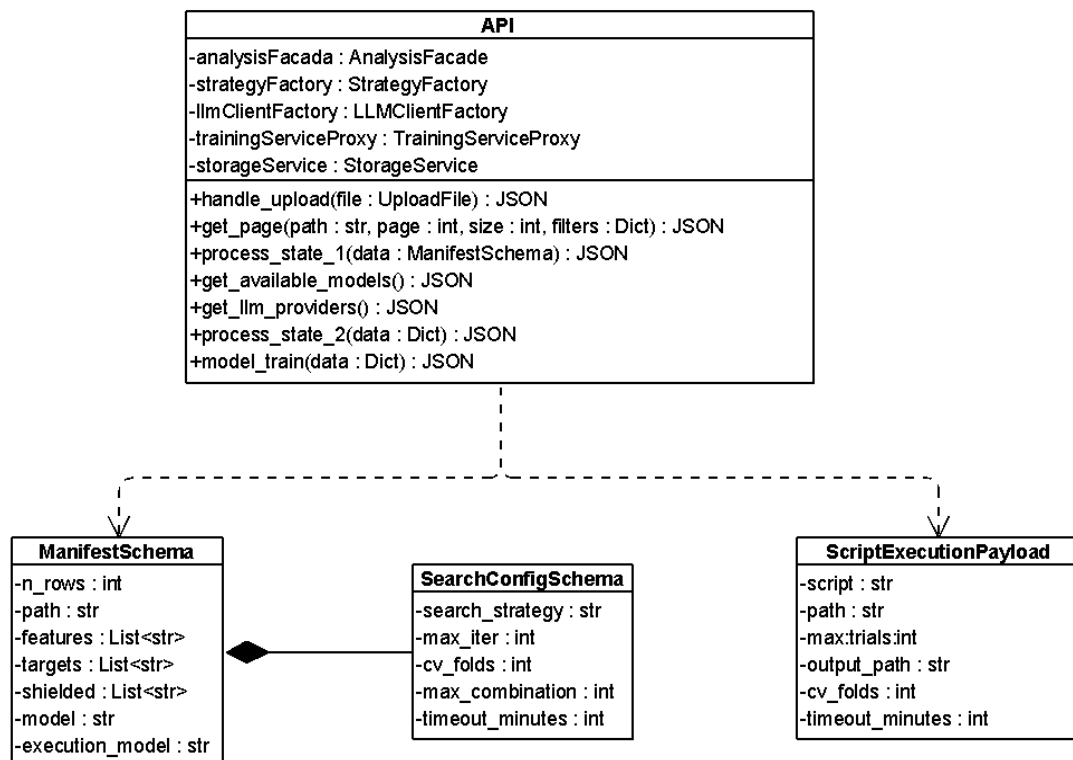


Figura 8. Diagrama de la clase API y los modelos de datos

Como se puede observar en el diagrama de la Figura 8, la API proporciona diferentes endpoints REST para facilitar la comunicación entre el *Front-End* y *Back-End*. A continuación, se va a detallar cada uno de los endpoints con una explicación del flujo principal que realizan: (i) */upload* encargado de la ingesta y el guardado inicial del dataset, (ii) */process-state1* inicia el análisis de las variables seleccionadas del dataset para clasificarlas mediante cálculos matemáticos y el uso de un modelo LLM para las subclases categóricas, (iii) */process-state2* utilizado para la generación del script de entrenamiento y (iv) */send-script* ejecuta el script de entrenamiento con las métricas seleccionadas por el usuario. Además de los endpoints asociados al flujo principal, se implementaron otros tres: */models* y */llm/providers*, orientados a realizar consultas y disponibilidad de los modelos y */get-page*, utilizado para la paginación del dataset en la primera etapa de selección de roles. Además, se garantiza la integridad de los datos enviados antes de la inyección en el sistema mediante esquemas Pydantic (*ManifestSchema*, *ScriptExecutionPayload*) para evitar que la API procese diccionarios genéricos. Esto hace que, antes de enviarse, se validen estrictamente las cargas de datos a enviar.

5.3.2. Clase *AnalysisFacade*

En la Figura 9, el componente de análisis orquesta, mediante un patrón de fachada, el preprocesamiento matemático y la preparación de datos para el modelo LLM.

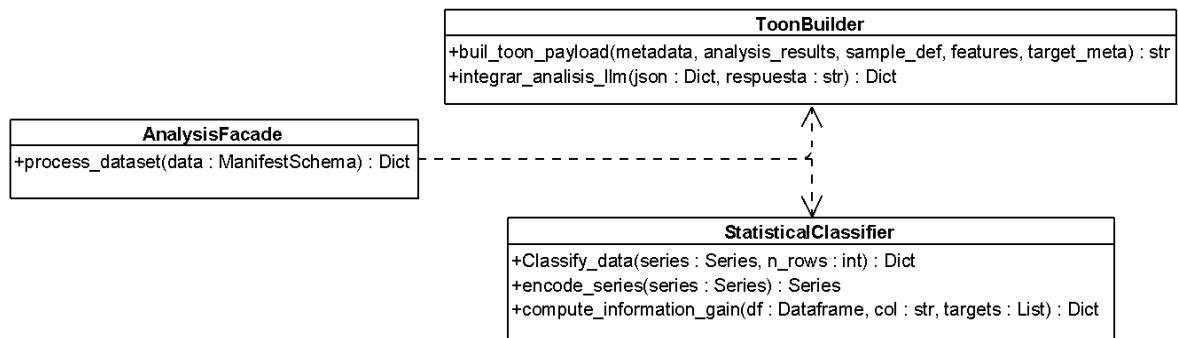


Figura 9. Diagrama de las clases de *AnalysisFacade*

La clase *AnalysisFacade* actúa como punto de entrada único que coordina dos procesos clave: por un lado, utiliza el *StatisticalClassifier* para calcular el Nivel de Entropía y la Ganancia de Información (Information Gain) de las columnas del dataset en el primer apartado y por otro lado, emplea el *ToonBuilder* para empaquetar los resultados puramente matemáticos junto con los metadatos del usuario en un formato semántico optimizado (*TOON Dossier*) que posteriormente será interpretado por el LLM previamente seleccionado por el usuario.

5.3.3. Motor de Orquestación de Estrategias

En la Figura 10, el motor de orquestación se encarga de determinar la mejor estrategia mediante una matriz de dependencias y un patrón *strategy* con el conjunto de todas las opciones a escoger.

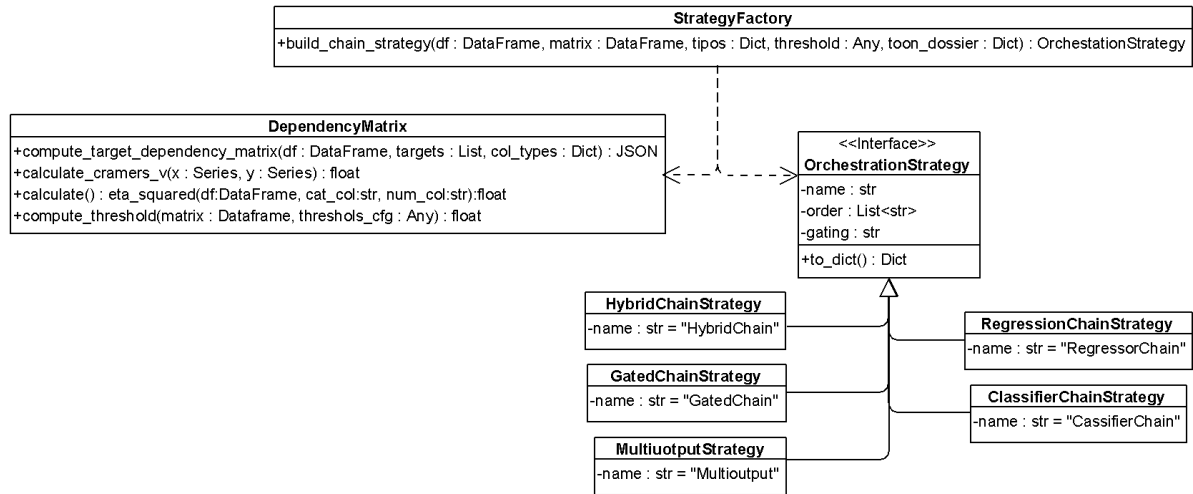


Figura 10. Diagrama de clases de OrchestrationStrategy

El *StrategyFactory* evalúa la matriz de dependencia de los targets apoyándose en la *DependencyMatrix* (que implementa cálculos como la V de Cramér y el eta cuadrado, según el tipo de clasificación de cada variable). Según el grado de correlación detectado, el patrón *Factory* instancia una topología concreta que implementa la interfaz *OrchestrationStrategy*. Las implementaciones disponibles (*GatedChainStrategy*, *MultiOutputStrategy*, *ClassifierChainStrategy*, *RegressionChainStrategy* y *HybridChainStrategy*) definen si existe un orden para realizar el entrenamiento y los mecanismos de bloqueo condicional (gating) entre modelos y se encuentran ausencias estructurales.

5.3.4. Gestor de Modelos de Language (LLM)

En la Figura 11, el sistema de integración con LLMs abstrae la comunicación con los motores de Inteligencia Artificial Generativa.

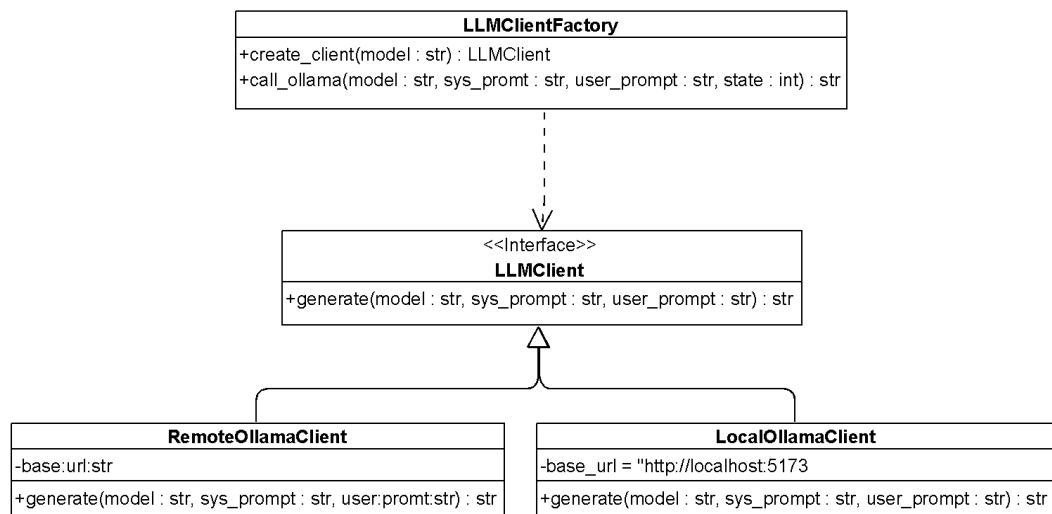


Figura 11. Diagrama de clases de LLMClientStrategy

Para garantizar el bajo acoplamiento, la clase *LLMClientFactory* genera clientes dinámicos que cumplen con el contrato de la interfaz *ILLMClient*. Esto permite al sistema alternar de forma transparente entre un despliegue de inferencia local (*LocalOllamaClient*) o un proveedor externo (*RemoteOllamaClient*) sin modificar la lógica de negocio central, encargándose de inyectar el System Prompt y procesar la salida del modelo.

5.3.5. Proxy de Entrenamiento y Laboratorio

En la Figura 12, el componente de entrenamiento aísla el entorno de generación y evaluación de los modelos del servidor principal de las variables.

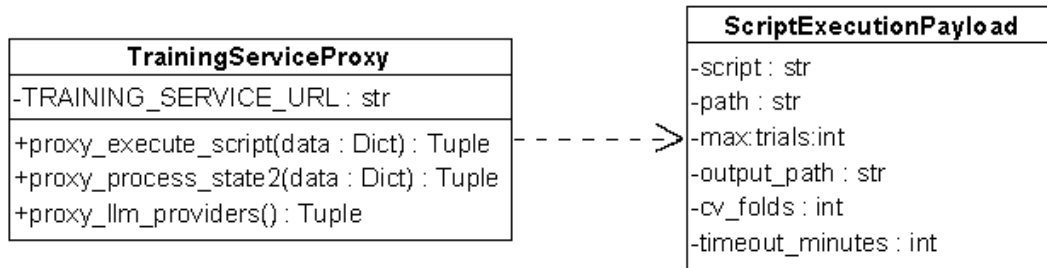


Figura 12. Diagrama de clases de TrainingServiceProxy

A diferencia de los sistemas monolíticos, SPECTRA delega la carga computacional intensiva en una imagen de contenedor Docker externa. La clase *TrainingServiceProxy* empaqueta el código fuente de orquestación (generado por el ILM) en un dataframe con los datos para inyectarlos en un objeto *ScriptExecutionPayload* y lo transmite mediante HTTP POST al entorno aislado o a la generación del código. Este diseño garantiza que las caídas por falta de memoria (OOM) o los tiempos de espera prolongados del algoritmo de Optuna no bloqueen los hilos asíncronos de la API principal.

5.4 Diagramas de secuencia

Para comprender el comportamiento secuencial del sistema y la comunicación entre microservicios, se presentan varios diagramas de secuencia que ilustran las interacciones asíncronas principales del sistema SPECTRA.

5.4.1 Diagrama de secuencia general

La Figura 21, añadida al Anexo B para una mejor visualización, muestra el flujo de interacción completo entre el usuario y el sistema.

El diagrama ilustra el ciclo de vida completo de un proyecto de AutoML. Este comienza con la carga del dataset por el usuario, seguida de la configuración de los roles de las variables y su clasificación por el motor estadístico. Después de generar la primera respuesta del modelo, se ejecuta el motor orquestador para seleccionar la mejor estrategia de entrenamiento. Tras la validación del Dossier Técnico, el sistema pasa a la generación de código y a la ejecución final del script de entrenamiento. Tanto la generación de código como la ejecución final se realizan en una imagen Docker para mayor seguridad, y cuenta con un bucle autocorrectivo en la etapa de generación del script para que el modelo LLM que el usuario haya escogido lo ejecute sin errores.

5.4.2 Diagrama de secuencia de Análisis y Orquestación

La Figura 22, incluida en el Anexo B para una mejor visualización, detalla el proceso secuencial en el que el servidor realiza la extracción matemática, el análisis semántico por parte del modelo LLM y el diseño de la estrategia de búsqueda de hiperparámetros, consolidando los resultados en un Dossier Técnico:

Este diagrama muestra cómo la clase *AnalysisFacade* coordina toda la clasificación mediante el análisis estadístico, así como la orquestación de la estrategia de entrenamiento. Primero, delega el cálculo de la entropía en *StatisticalClassifier* y empaqueta los resultados en un archivo TOON con el *ToonBuilder*. Tras obtener la clasificación de las subclases de las variables categóricas del LLM (*LLMClientFactory*), el patrón *facade* invoca a *DependencyMatrix* para evaluar la correlación entre los objetivos predictivos. Finalmente, el *StrategyFactory* determina la estrategia óptima (p. ej., *GatedChainStrategy*) a partir de la matriz generada previamente, autogenera el código fuente de la topología y encapsula todo en el Dossier Técnico que la API devuelve al *Front-End* para que el usuario lo visualice.

5.4.3 Diagrama de secuencia del Laboratorio AutoML

La Figura 23, añadida en el Anexo B para una mejor visualización, detalla el proceso completo de generación dinámica del código de entrenamiento para los modelos de ML y su posterior ejecución aislada, destacando la interacción del usuario como supervisor del proceso.

El diagrama ilustra el flujo de trabajo con mayor complejidad que todos los demás, donde se divide en tres fases críticas:

La *autogeneración y corrección autónoma (Self-Healing)*, donde el sistema entra en un bucle de autocorrección tras que el usuario valide la entrada de los datos del dossier de la etapa anterior, ya clasificado. El sistema ejecuta el script de forma aislada y, si se produce un fallo (error de compilación o de ejecución), extrae la traza del error y consulta al LLM en modo *debugger*. El modelo de lenguaje corrige el código dinámicamente y lo vuelve a enviar al *Sandbox*. Este ciclo se repite de forma autónoma hasta que la ejecución sea completamente exitosa.

Una vez que el script esté correcto, se le envía al usuario para que garantice la integridad de las métricas seleccionadas, otorgándole la autoridad de auditoría antes de proceder con el entrenamiento.

Por último, está el *Entrenamiento Definitivo*, que ejecuta el script de manera aislada con una optimización exhaustiva de los hiperparámetros que el usuario estableció en la etapa anterior. Esto también incluye, al finalizar, la persistencia de los modelos en formato *.joblib* y la devolución de un *Dashboard* con las métricas de cada uno de ellos, guardado en la misma carpeta que los modelos en formato *.json*.

6. Gestión de datos e información

En esta sección se presenta la estrategia de gestión de datos implementada en el sistema, abordando las consideraciones de privacidad y seguridad, así como la estructura y organización de la información almacenada.

6.1. *Enfoque de gestión de privacidad*

El sistema se ha implementado bajo el principio fundamental de la privacidad por diseño. En lugar de centralizar los conjuntos de datos en plataformas en la nube de terceros, la persistencia, la ingesta y el entrenamiento de los modelos nunca abandonan el entorno de ejecución local.

La arquitectura adopta un modelo híbrido descentralizado en el que toda la carga de cómputo de los datos permanece en el entorno local, mientras que la lógica y el orquestador de código pueden delegarse tanto en el entorno local como en un servidor remoto. Para garantizar la confidencialidad, los datos clínicos nunca se delegan en un servidor remoto, sino que permanecen siempre en el entorno local. Esto hace que el entorno remoto se limite al envío de un manifiesto estructurado de metadatos con el que se imposibilita cualquier tipo de persistencia, reconstrucción o ingeniería inversa en el nodo remoto.

Esta arquitectura local-remota presenta unas ventajas significativas desde un punto de vista de la seguridad informática y la protección de datos. Al mantener el procesamiento dentro de la infraestructura interna, se minimiza el riesgo asociado a la transmisión de datos y se simplifica el cumplimiento de la normativa de protección de datos, en especial del Reglamento General de Protección de Datos (RGPD) de la Unión Europea.

Finalmente, para validar la viabilidad y el rendimiento del proyecto en un escenario operativo real, las fases de prueba se han realizado con un conjunto de datos proporcionado por el Complejo Hospitalario Universitario de Ourense (CHUO). En estricto cumplimiento de los artículos del RGPD que regulan el tratamiento de datos personales con fines de investigación científica, dichos datos fueron sometidos a un proceso previo y riguroso de anonimización irreversible, eliminando cualquier información que pudiera identificar a los pacientes.

6.2. *Datos almacenados por el sistema*

SPECTRA está diseñado para ser un sistema con un almacenamiento persistente mínimo, lo que garantiza su ligereza. Los únicos datos e información que se guardan de forma permanente son:

Ficheros de Configuración Dinámica. Almacenan temporalmente el diccionario tipado en el que se mapean los roles de las variables seleccionadas.

Dossier Semántico Transitorio. Un fichero estructurado en formato Json que contiene las métricas estadísticas de las variables, así como la inyección de la respuesta del LLM de la etapa de clasificación.

Modelos de Entrenamiento de las Variables. Artefactos de aprendizaje automático serializados por cada variable a predecir seleccionada por el usuario bajo una nomenclatura con formato .joblib.

Archivo Resultado de cada Modelo. Consta de las métricas utilizadas para entrenar el modelo seleccionado por el LLM y de sus respectivos valores de confianza, para evaluar si la predicción es óptima o no.

Todos estos datos no contienen información alguna que permita identificar a los pacientes, ya que solo se almacenan las métricas de cada modelo.

6.3. Estructura de directorios

El sistema organiza la información en una estructura de directorios diseñada para facilitar la gestión y el acceso a los componentes. A continuación, se detallan los principales directorios del servidor:

backend/uploads. Espacio reservado para la gestión física de los conjuntos de datos. Este aloja los archivos cargados por el usuario (en formatos .csv, .xlsx o .parquet) que sirven de entrada para entrenar las variables que el usuario seleccione. Estos archivos se guardan temporalmente, una vez que se hayan entrenado todos los modelos, se eliminarán.

backend/dossiers. En este directorio se almacenan los archivos en formato JSON y los TOON Dossier. Estos documentos contienen tanto el perfilado estadístico como las características de salida del LLM y sirven de base conceptual tanto para el orquestador como para la elaboración del script de modelado.

backend/models/scripts. Contiene el código generado automáticamente por el LLM. Este archivo se incluye en este directorio una vez generada la primera respuesta, de modo que se reedite cada vez que se vuelvan a realizar respuestas de corrección de código.

backend/models. En este directorio se guardan y persisten los modelos predictivos en formato .joblib. Estos archivos se generan automáticamente al finalizar la ejecución del script. También se incluyen archivos JSON que especifican las métricas utilizadas por cada modelo, junto con sus valores de confianza, para mostrárselos al usuario final.

7. Aplicación a datos reales

El sistema SPECTRA implementa un flujo de trabajo AutoML completo y autónomo, desde la recepción de datos hasta la exportación de los modelos entrenados, listos para su uso. Como ya se ha descrito, este proceso se apoya en una arquitectura distribuida que combina motores estadísticos locales, una orquestación semántica basada en modelos de LLM y entornos de ejecución aislados para el entrenamiento.

En este apartado se describen los experimentos realizados para evaluar el rendimiento del sistema a partir de datos reales del servicio de radiología. El objetivo principal es determinar la viabilidad y la eficacia de SPECTRA para identificar la estrategia, el diseño de patrones y la clasificación de los datos según la especialidad, la sala y la prioridad.

Para la realización de estos experimentos se ha utilizado un dataset anonimizado cedido por el Complejo Hospitalario Universitario de Ourense (CHUO), previamente clasificado manualmente por especialistas y corregido ortográficamente por el cotutor de este proyecto. Este dataset tiene campos de texto libre (descripción, datos clínicos, sospecha de diagnóstico) y campos numéricos (edad) que el sistema AutoML debe procesar sin intervención manual.

7.1. Descubrimiento de Datos y Perfilado

A diferencia de los enfoques tradicionales, en los que el científico de datos programa manualmente la limpieza y la extracción de características, SPECTRA asume el control del flujo desde el primer instante mediante la clasificación matemática de los datos inyectados en un Dossier. Este flujo se organiza en tres etapas principales.

7.1.1. Ingesta y Definición de Roles

El proceso comienza con la entrada de datos en el documento. Su función principal es procesar diversos formatos de documentos (Excel, CSV o Parquet) para garantizar la integridad de los datos y visualizarlos para su posterior selección de roles.

Durante la configuración inicial, se definen las variables clave para la creación del entrenamiento. En este caso, los datos constan de tres columnas target designadas como especialidad, sala y prioridad; y cuatro columnas features designadas como descrip, edad, datosclini y sospechadiag. Como la variable edad no suele aportar mucha información a los modelos de LLM, la suelen evadir; se debe blindar, ya que es de suma importancia para la obtención de los targets.

Tras la configuración, se debe seleccionar el modelo a aplicar para subclasificar las variables categóricas. Como no todos los modelos dan la respuesta correcta, sólo se mostrarán los que no están pensados para el código.

7.1.2. Motor Estadístico y Análisis Técnico

Una vez definidos los roles en la interfaz, se ha procesado mediante un motor estadístico local utilizando las librerías vectorizadas como *pandas* y *scikit-learn*. El objetivo de esta fase es extraer un perfil físico a partir de los datos sin comprometer la privacidad del paciente, es decir, obtener métricas abstractas sin enviar la información clínica a servidores externos. El análisis técnico se dividió en tres pasos clave.

El primero de ellos es el *Perfilado Estructural y la Detección de Tipos*. La función *classify_data()* examina la topología física de la columna (p. ej., textos clínicos libres o valores numéricos de edad), calculando ratios de unicidad, porcentaje de valores nulos y asimetría. Gracias a esto, detecta automáticamente la cardinalidad del dato.

Por otro lado, se cuenta con la Gestión de Restricciones Médicas (Shielded Features). El sistema verifica si el usuario ha marcado la variable como "blindada" o "intocable" e inyecta instrucciones prioritarias directamente al orquestador del LLM.

Por último, se tiene el Cálculo de Tracción Predictiva. Mediante la función *compute_information_gain()*, el motor calcula la Información Mutua (Mutual Information) para cuantificar el peso estadístico real que tiene la variable sobre los Targets clínicos, discriminando el ruido de los patrones útiles.

Este proceso de evaluación secuencial queda reflejado en el núcleo del iterador de la clase *AnalysisFacade*:

```
01 for col in data.features:
02     jseries = df[col]
03     col_data = classify_data(jseries, n_rows)
04     update_data = {
05         "user_mandatory": col in data.shielded,
06         "unique_pool": series.dropna().astype(str).unique()
07         [:20].tolist() }
08     if col in data.shielded:
09         update_data["llm_instruction"] = (
10             "This feature is statistically irrelevant, but the user
11             requires its inclusion "
12             "for business reasons. Generate code that includes it in
13             every stage of the pipeline." )
14     col_data.update(update_data)
15     is_discrete = col_data["technical_level"] in [1, 2]
16     col_data["information_gain"] = compute_information_gain(
17         df, col, data.targets, target_meta, is_discrete,
18         device=device )
19     analysis_results[col] = col_data
```

Un aspecto crucial de este módulo se trata de la optimización del rendimiento en la generación del payload que viajará al LLM como un prompt informativo. Al enfrentarse a miles de registros médicos, SPECTRA implementa la función *stratified_sample_100()*, que extrae un subconjunto estadísticamente representativo de los datos (*sample_df*) manteniendo intacta la distribución de las clases objetivo.

```
01 sample_df = stratified_sample_100(df[columnas], data.targets)
02 sample_df_clean = sample_df.astype(object).fillna("")
03 metadata = {"n_rows": n_rows, "targets": data.targets}
04 user_prompt = build_toon_payload(metadata, analysis_results,
05     sample_df_clean, data.features, target_meta)
```

La encapsulación de este proceso en la función *build_toon_payload()* inyecta todos los cálculos matemáticos descritos (cardinalidad, Information Gain, muestras), así como las indicaciones para el sistema del modelo LLM, en un único documento JSON estructurado. Este mecanismo permite aislar temporalmente el trabajo computacional intensivo, reduciendo drásticamente los tokens necesarios y garantizando que el orquestador del LLM de la siguiente etapa reciba únicamente la metainformación esencial para redactar el script de aprendizaje automático.

7.1.3. Clasificación Semántica mediante LLM

Uno de los componentes más innovadores y complejos aplicados a este caso de uso médico fue la inferencia semántica mediante inteligencia artificial generativa. A diferencia del motor estadístico (que solo observa números y metadatos), el orquestador local de SPECTRA (gobernado por la API de Ollama) tiene la responsabilidad de razonar sobre la naturaleza lingüística y lógica de las variables clínicas categóricas.

Para garantizar que las inferencias del Modelo de Lenguaje (LLM) fuesen estructuradas, deterministas y libres de alucinaciones, el sistema implementa una técnica de inyección de contexto rigurosa (*Zero-Shot Prompting*). En la clase *AnalysisFacade*, se define un System Prompt estricto que instruye al modelo sobre cómo debe clasificar la información, exigiendo una salida formateada.

Al inyectar el TOON Dossier (que incluye la muestra representativa de pacientes) como contexto del usuario (*user_prompt*), el modelo fue capaz de observar la relación entre los datos y razonar lógicamente. Para el dataset del CHUO, el LLM determinó que la columna especialidad era de naturaleza NOMINAL (categorías médicas sin jerarquía), mientras que la columna prioridad fue clasificada correctamente como ORDINAL (con un orden de gravedad subyacente).

Dado que los LLMs generan texto libre por naturaleza, extraer los datos estructurados vitales para el aprendizaje automático requirió un paso de posprocesamiento. Para ello, SPECTRA delega la limpieza a la función *integrar_analisis_llm()*, la cual emplea expresiones regulares (Regex) para capturar con precisión milimétrica tres campos: la subclase inferida, el mapeo numérico jerárquico y el razonamiento textual del modelo.

```

01 def integrar_analisis_llm(json_tecnico, respuesta_llm):
02     patron = r"(\w+):\s*subclass:\s*(\w+)\s*mapping:\s*"
03         ([^\n+])\s*reasoning:\s*([^\n+])"
04     hallazgos = re.findall(patron, respuesta_llm)
05     for nombre, subclass, mapping, reasoning in hallazgos:
06         val_mapping = mapping.strip()
07         final_mapping = None if val_mapping.lower() == "null" else
08             val_mapping
09         if nombre in json_tecnico.get("categorical_evaluation", {}):
10             item = json_tecnico["categorical_evaluation"][nombre]
11             item["subclass"] = subclass.strip()
12             item["mapping"] = final_mapping
13             item["reasoning"] = reasoning.strip()
14     return json_tecnico

```

Gracias a este proceso de parsing, el mapeo jerárquico deducido por la IA (p. ej., asociando C=2, B=1, A=0 para respetar la gravedad en la urgencia) se inyecta directamente en el objeto JSON técnico. Esta traducción automática del lenguaje natural a reglas matemáticas es vital para que, en la etapa posterior de generación de código, los algoritmos ensamblados (*XGBoost*, *RandomForest*) procesen los historiales clínicos respetando el contexto médico original, algo que los sistemas de codificación tradicionales (Label Encoding ciego) no podrían lograr.

7.2. Orquestación Autónoma y Entrenamiento

Tras completar con éxito el perfilado estadístico y semántico, el sistema se enfrentaba al mayor desafío arquitectónico del proyecto: decidir, sin intervención humana, la estrategia de ensamblado óptima para abordar un problema de clasificación con múltiples targets (Multi-Output). En el ámbito de los historiales clínicos, los datos son inherentemente propensos a presentar vacíos estructurales y sesgos por omisión (Missing Not At Random, o MNAR). Para poder realizar predicciones fiables y evitar que

el ruido estadístico de los valores nulos corrompiera los modelos, SPECTRA debía evaluar las dependencias entre los Targets, diseñar un flujo de entrenamiento condicional lógico y delegar en el LLM la redacción del código de aprendizaje automático, que se ejecutaría posteriormente de forma segura en un entorno virtualizado.

7.2.1. Evaluación de Dependencias y Estrategia

La toma de decisiones algorítmicas en SPECTRA no se basa en heurísticas estáticas, sino en un análisis matricial dinámico. Tras el perfilado, el sistema debía decidir la estrategia óptima para abordar un problema complejo de múltiples salidas (Multi-Output) en un dominio clínico inherentemente propenso a valores nulos.

Para lograrlo, la clase AnalysisFacade invoca el cálculo de la covarianza cruzada mediante la función `compute_target_dependency_matrix()`. Al analizar las relaciones entre especialidad, sala y prioridad, el sistema detectó ausencias estructurales lógicas (casos de Missing Not At Random, o MNAR). Ante este escenario, la función evaluadora `build_chain_strategy()` descartó automáticamente un enfoque predictivo plano y propuso una arquitectura anidada de tipo GatedChain (o cadena condicional).

```
01 df_for_matrix = df.sample(n=min(10000, len(df)), random_state=42)
02     matriz =compute_target_dependency_matrix(df_for_matrix,
03         data.targets,json_tecnico["targets_evaluation"])
04     threshold_config = getattr(data,'threshold_cfg', 'auto')
05     orquestation = build_chain_strategy(df,matriz,
06         json_tecnico["targets_evaluation"],
07         threshold_config, json_tecnico)
```

Esta estrategia divide el problema: aísla las variables condicionales y entrena submodelos independientes para los conjuntos dependientes, garantizando que el ruido generado por las celdas vacías (p. ej., un paciente sin prioridad porque su prueba no lo requiere) no contamine el aprendizaje de las redes neuronales posteriores.

7.2.2. Generación de Código y Auditoría

Con base en el plan orquestado (*orchestration plan*) y en el TOON Dossier, el orquestador LLM redactó de forma autónoma el script de entrenamiento en Python. El modelo generativo fue capaz de importar las librerías necesarias de inferencia (*scikit-learn*, *xgboost*, *optuna*) y construir pipelines de procesamiento NLP, integrando componentes como el *TfidfVectorizer* para convertir el texto libre de los volantes en matrices matemáticas asimilables.

Es necesaria la validación del usuario del script generado mostrando el código fuente al usuario en el *Front-End*. Durante esta etapa interactiva, se configuraron restricciones computacionales estrictas para garantizar la reproducibilidad y contener el gasto de recursos:

- Validación cruzada de 10 pliegues (10-Fold CV).
- Límite de 50 intentos exploratorios (trials) para la optimización bayesiana con la librería Optuna.
- Un tiempo límite (Timeout) de seguridad establecido en 60 minutos.

7.2.3. Ejecución en Sandbox Virtualizado

Una vez auditado y aprobado el script, el núcleo principal (*Back-End* de FastAPI) no ejecuta el entrenamiento en su propio hilo de proceso por motivos de seguridad y estabilidad. En su lugar,

SPECTRA delega esta tarea en un microservicio aislado (*Sandbox*), implementado como un contenedor Docker independiente basado en la imagen `python:3.12-slim` (ghcr.io/jorgercid/spectra-training-service).

Esta comunicación interprocesos se gestiona mediante el módulo `training_service_proxy.py`. El servidor principal inyecta el script y los hiperparámetros como un payload JSON mediante una petición HTTP POST segura hacia el puerto expuesto del contenedor *Sandbox*:

```
01 def proxy_execute_script(data: dict[str,Any]) ->
02     tuple[dict[str,Any], int]:
03     url = f"{TRAINING_SERVICE_URL}/process-state2/execute-script"
04     try:
05         response = requests.post(url,son=data,
06             timeout=TRAINING_SERVICE_TIMEOUT)
07         return response.json(),response.status_code
08     except requests.exceptions.Timeout:
09         logger.error("Training service timed out after %ss",
10             TRAINING_SERVICE_TIMEOUT)
11         return {"status": "error", "message": "Timeout exceeded"},
12             504
```

Al operar en este entorno aislado, el sistema exploró dinámicamente el espacio de hiperparámetros de múltiples algoritmos competitivos, buscando maximizar la métrica F1-Score general, sin riesgo de corromper el servidor principal ni interrumpir automáticamente el proceso mediante el manejador de Timeout si se exceden los límites establecidos.

7.3. Resultados del Entrenamiento AutoML

Tras la ejecución en paralelo en el contenedor Docker (*Sandbox*), SPECTRA persistió los resultados y los modelos entrenados. A diferencia de las aproximaciones manuales tradicionales, esta sección presenta la evaluación consolidada del rendimiento predictivo generado de forma autónoma por un LLM orquestador en las tres tareas definidas: (i) identificación de la especialidad, (ii) determinación de la sala, y (iii) asignación de prioridad clínica.

La evaluación de los modelos ganadores se fundamenta en métricas derivadas de la matriz de confusión [D14], complementadas con el coeficiente Kappa de Cohen [D15]. Estas medidas proporcionan una visión integral del rendimiento predictivo.

7.3.1. Rendimiento Global

La matriz de confusión constituye la base para el cálculo de las principales métricas de evaluación. En el contexto específico de la clasificación de volantes médicos, esta matriz se interpreta de la siguiente manera para cada una de las tareas de clasificación:

Tabla 3. Estructura teórica de una matriz de confusión binaria

	Real: Positive	Real: Negative
Predicción: Positive	True Positive (TP)	False Positive (FP)
Predicción: Negative	False Positive (FP)	True Negative (TN)

A partir de esta matriz se pueden extraer diferentes propiedades que permiten mejorar la comprensión de los resultados extraídos por SPECTRA:

- *Exactitud* (Accuracy): Representa la proporción de clasificaciones correctas respecto al total de predicciones realizadas.
- *Precisión* (Precision): Indica la proporción de volantes correctamente clasificados entre todos los asignados a una clase concreta. Esta métrica es especialmente relevante para evaluar la confiabilidad de las asignaciones positivas.
- *Puntuación F1* (F1-Score): Permite medir el promedio armónico ponderado de la precisión y el recall. Es la métrica base utilizada por la optimización bayesiana (Optuna) del *Sandbox*.
- *Coeficiente Kappa de Cohen*: Evalúa la concordancia entre las clasificaciones del modelo y las etiquetas reales, ajustada por la probabilidad de acierto al azar.

7.3.2. Métricas detalladas por Objetivo

Una vez finalizado el entrenamiento mediante el script autogenerado, el sistema reportó el rendimiento de los algoritmos competitivos evaluados en el espacio de búsqueda. El objetivo de estos experimentos automáticos fue determinar qué algoritmo ofrecía el mejor rendimiento en cada nodo de la arquitectura GatedChain.

Clasificación de Especialidad (MSQ vs OTHER)

La Tabla 4 muestra los resultados de Validación Cruzada (10-Fold CV) obtenidos para la tarea de determinar la especialidad médica del dataset, mediante el análisis del texto de las variables seleccionadas como features.

Tabla 4. Resultados de la clasificación de especialidad (MSQ vs NO-MSQ)

	CV Scor	Accuracy	F1	Kappa
Ejecución 1	0.9675	0.9679	0.9676	0.8964
Ejecución 2	1.0000	1.0000	1.0000	1.0000

Como se puede observar, el modelo alcanzó un rendimiento estructuralmente perfecto (100% en todas las métricas) en casi todos los casos de ejecución. Esto demuestra que, para esta variable en concreto, la terminología libre empleada por los médicos es altamente determinista. El sistema seleccionó hiperparámetros óptimos de *max_depth*: -1 y *n_estimators*: 500 para sellar este nodo del ecosistema.

Clasificación de Sala (*H* vs *NH*)

La Tabla 5 muestra los resultados de la clasificación de volantes condicionados, determinando si requieren un estudio de hombro (H) o no (NH).

Tabla 5. Resultados de la clasificación de sala (H vs NH)

	CV Scor	Accuracy	F1	Kappa
Ejecución 1	0.91291	0.91288	0.91291	0.82596
Ejecución 2	0.8164	0.8604	0.8603	0.7204

En esta tarea, el motor AutoML finalizó la mejor ejecución de optimización, alcanzando un F1-Score de 0.913 (91.29%) y una exactitud (Accuracy) paralela de 91.28%. Este alto rendimiento global, respaldado además por una fuerte concordancia estadística (índice Kappa de 0.826), confirma la solidez de la arquitectura en cascada generada dinámicamente por el orquestador. Estos resultados validan la capacidad del algoritmo seleccionado para discernir con precisión los patrones del lenguaje clínico, evitando de forma robusta que el modelo se confunda y clasifique erróneamente patologías que no requerían estudio.

Clasificación de Prioridad Clínica (*A*, *B*, *C*)

Finalmente, la Tabla 6 presenta los resultados de la clasificación de prioridad general, un problema multiclase ordinal abordado directamente.

Tabla 6. Resultados de la clasificación de prioridad (A, B o C)

	CV Scor	Accuracy	F1	Kappa
Ejecución 1	0.7199	0.7272	0.7201	0.5482
Ejecución 2	0.7049	0.7507	0.7465	0.5851

En esta tarea, indudablemente la más compleja del ecosistema clínico, el motor del sistema completó la optimización con un F1-score de 0.7201 (72.01%) y una exactitud de 72.72% en la mejor ejecución. Aunque es el valor más bajo de los tres objetivos, este rendimiento refleja fielmente la subjetividad inherente a los triajes médicos humanos. La concordancia moderada (Kappa 0.585) mejoró significativamente gracias al mapeo ordinal ($A=0$, $B=1$, $C=2$) que el LLM generó automáticamente para la clasificación semántica de las subclases categóricas.

7.4. Conclusiones

A partir de los resultados extraídos del sistema SPECTRA, se pueden extraer las siguientes conclusiones:

El predominio de entrenamientos mediante Gradient Boosting: El espacio de búsqueda de hiperparámetros de SPECTRA demostró, sin sesgo humano, que *XGBClassifier* superó considerablemente a otros ensamblados clásicos, como Random Forest, en el procesamiento de historias clínicas vectorizadas.

Validación de la Arquitectura Condicional: La decisión autónoma del LLM de implementar un modelo GatedChain para resolver ausencias estructurales (MNAR) resultó exitosa, logrando un F1-Score de 87.02% en el subnodo de la Sala, sin contaminar el entrenamiento general.

Los resultados validan que la arquitectura AutoML Distribuida de SPECTRA no solo es viable frente al desarrollo de Jupyter Notebooks manuales, sino que acelera exponencialmente la democratización y puesta en producción de soluciones de Inteligencia Artificial para el sector salud.

8. Pruebas llevadas a cabo

En este apartado, se detallan las pruebas realizadas para verificar la correcta implementación de las funcionalidades definidas en las historias de usuario. Para ello, se han realizado comprobaciones mediante pruebas de caja negra, que verifican la funcionalidad sin considerar la estructura interna del código, centrándose únicamente en las entradas de datos y las salidas obtenidas.

Las pruebas ejecutadas se diseñaron específicamente para validar estrictamente el cumplimiento de los criterios de aceptación de las once historias de usuario definidas en la sección 4. Cabe destacar que se han omitido pruebas redundantes, flujos intermedios idénticos y validaciones sintácticas menores que no aportan información adicional a la automatización de la infraestructura.

Para la ejecución de las pruebas se utilizó un dataset de volantes corregidos y anonimizados, cedido por el CHUO, previamente clasificado por especialistas según el tipo de especialidad a la que pertenece, la sala y la prioridad. Este dataset presenta un escenario multiobjetivo realista compuesto por múltiples variables de diversas naturalezas semánticas, desequilibrio de clases y ausencias estructurales.

En el Anexo C, se muestra el conjunto de pruebas realizadas para verificar el cumplimiento de cada historia de usuario.

9. Manual de usuario e instalación

En este apartado se han elaborado los manuales de usuario y de uso, junto con los requisitos mínimos de software. La creación de dichos manuales será útil para el correcto funcionamiento e instalación de la aplicación en el equipo a instalar.

9.1. *Requisitos mínimos*

El proyecto ha sido optimizado para ejecutarse en equipos con los sistemas operativos Windows 11, Ubuntu 22.04 o distribuciones Linux similares.

Para ejecutar el Front-End del proyecto, es necesario contar con Node.js v22.18 o superior y con el gestor de paquetes npm. Este componente se ha desarrollado con React 19.0 y TypeScript, por lo que las dependencias necesarias se instalarán automáticamente mediante el gestor de paquetes.

Para el componente *Back-End*, es necesaria la instalación de *Python 3.12* y del paquete *python3.12-venv* para crear y gestionar el entorno virtual necesario para ejecutar el programa. Este entorno permite aislar las dependencias analíticas de *scikit-learn* y *pandas*, evitando conflictos con las dependencias de otros proyectos que el sistema pueda tener. Las dependencias específicas del proyecto están documentadas en el archivo *requirements.txt*, listo para realizar la instalación completa de las dependencias. Además, será necesario tener instalado *Docker 29.0* y si se quiere probar un modelo de LLM en concreto, también se podrá instalar *Ollama*.

9.2. *Manual de instalación*

A continuación se detallan las instrucciones de instalación para poner en funcionamiento el sistema tanto en entornos Windows como en Linux. El sistema requiere la instalación de varias tecnologías, incluyendo *Node.js*, *Python* y *Docker*, seguida de la configuración de los componentes y de la ejecución mediante los scripts proporcionados en el código.

9.2.1. *Instalación en Windows*

Para la instalación y uso de SPECTRA, es necesario tener instalados los siguientes componentes:

Node.js (versión 22.18 o superior): descargar o instalar desde la página principal de Node.js¹.

Python 3.12: descargar o instalar desde la página oficial de Python². Durante la instalación, asegúrese de marcar la opción “Add Python to PATH”.

Docker 29.0: descargar o instalar desde la página oficial de Docker³ asegurándose de tener habilitado WSL2.

Ollama: descargar o instalar desde la página principal de Ollama⁴ y descargar algún modelo base (*ollama run llama3*).

Instalación del sistema

1. Descomprimir el archivo del proyecto en la ubicación deseada.
2. Abrir la ventana de *PowerShell* o el *símbolo del sistema* (*cmd*) con privilegios de administrador.

¹ Descargar Node.js: [Node.js — Download Node.js®](#)

² Descargar Python (3.12): [Python Releases for Windows | Python.org](#)

³ Descargar Docker (29.0): [Docker Desktop: The #1 Containerization Tool for Developers | Docker](#)

⁴ Descargar Ollama: <https://ollama.com/>

3. Verificar que las dependencias están correctamente instaladas ejecutando estos comandos:

```
01 node --version
02 python --version
03 docker --version
04 ollama --version
```

4. Navegar hasta la carpeta raíz del proyecto.
5. Asegurarse de que Docker Desktop esté abierto y en ejecución.
6. Asegurarse de que el script *scriptWindows.bat* tiene permisos de ejecución.
7. Ejecutar el sistema utilizando el siguiente comando:

```
01 .\scriptWindows.bat
```

El script realizará automáticamente las siguientes acciones (i) comprobará la conexión con Ollama y Docker, (ii) instalará la imagen base de Docker, (iii) creará el entorno virtual de Python e instalará las dependencias necesarias para el análisis de datos, (iv) lanzará el componente *Back-End*, (v) instalará las dependencias del *Front-End* y (vi) lanzará el servidor web.

Una vez completada la ejecución del proceso, se podrá acceder a la plataforma en la URL <http://localhost:5173>.

9.2.2. Instalación en Linux (Ubuntu 22.04)

Para la instalación del sistema en entornos Linux (Ubuntu 22.04), es necesario instalar los siguientes componentes:

Node.js (versión 22.18 o superior):

```
01 curl -fsSL https://deb.nodesource.com/setup_22.x
02 sudo apt install -y nodejs
```

Python 3.12:

```
01 sudo apt update
02 sudo apt install -y software-properties-common
03 sudo add-apt-repository ppa:deadsnakes/ppa
04 sudo apt update
05 sudo apt install -y python3.12 python3.12-venv
```

Docker 29.0:

```
01 sudo apt update
02 sudo apt install ca-certificates curl gnupg
03 sudo install -m 0755 -d /etc/apt/keyrings
04 curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo
   gpg --dearmor -o /etc/apt/keyrings/docker.gpg
05 sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

Instalación del sistema

1. Descomprima el archivo del proyecto en la ubicación deseada.
2. Abra una terminal y navegue hasta la carpeta raíz del proyecto.
3. Verifique que las dependencias están correctamente instaladas ejecutando los siguientes comandos

```
01 node --version
02 python --version
03 docker --version
03 ollama --version
```

4. Otorgue permisos de ejecución al script de inicio:

```
01 chmod +x scriptLinux.sh
```

5. Ejecute el sistema utilizando el siguiente comando:

```
01 ./scriptLinux.sh
```

El script realizará automáticamente las siguientes acciones (i) comprobará la conexión con Ollama y Docker, (ii) instalará la imagen base de Docker, (iii) creará el entorno virtual de Python e instalará las dependencias necesarias para el análisis de datos, (iv) lanzará el componente *Back-End*, (v) instalará las dependencias del *Front-End* y (vi) lanzará el servidor web.

Al igual que en Windows, se podrá acceder al sistema a través de la URL <http://localhost:5173> en su navegador web.

9.3 Manual de usuario

En las siguientes secciones se explica el paso a paso para la utilización de la aplicación y garantizar una experiencia de usuario fluida y satisfactoria.

9.3.1. Carga del documento

En la Figura 13 se muestra la pantalla de carga de la aplicación, que permite subir los archivos para configurar los roles de las variables. Se identifican los siguientes elementos:

1. **Área de arrastrar y soltar.** Permite subir archivos arrastrándolos a esta zona.
2. **Botón Deseleccionar.** Una vez seleccionado un archivo se permite eliminarlo ya seleccionado.
3. **Área para seleccionar.** Permite buscar y seleccionar un archivo desde el explorador.
4. **Botón Enviar.** Permite procesar el archivo seleccionado y avanzar al siguiente paso.



Figura 13. Manual de usuario - Selección de documento

9.3.2. Definición de Roles y Blindaje de Variables

En la Figura 14 se presenta la vista de la presentación del documento, en la que se muestran los datos en formato de tabla. Se identifican los siguientes pasos:

1. **Botón de volver.** Permite al usuario interrumpir el proceso y regresar a la pantalla anterior
2. **Botón Seleccionar modelo.** Permite seleccionar el modelo a implementar.
3. **Botón Mostrar columnas.** Permite seleccionar qué columnas se muestran en la tabla.
4. **Selector de rol - TARGET.** Cabecera interactiva que permite, con dos clics, seleccionar las variables targets.
5. **Filtrado de columna.** Permite buscar determinadas palabras en cada columna.
6. **Selector de rol - FEATURE.** Cabecera interactiva que permite mediante un clic la selección de las features.
7. **Blindado de columna.** Permite seleccionar una columna como blindada clicando el icono
8. **Módulo de Paginación.** Permite cambiar de página para ver todos los datos de la tabla mediante flechas de avance incremental, así como un input para seleccionar la página a visualizar.
9. **Botón Corregir.** Permite corregir las columnas de texto y avanzar al siguiente paso.

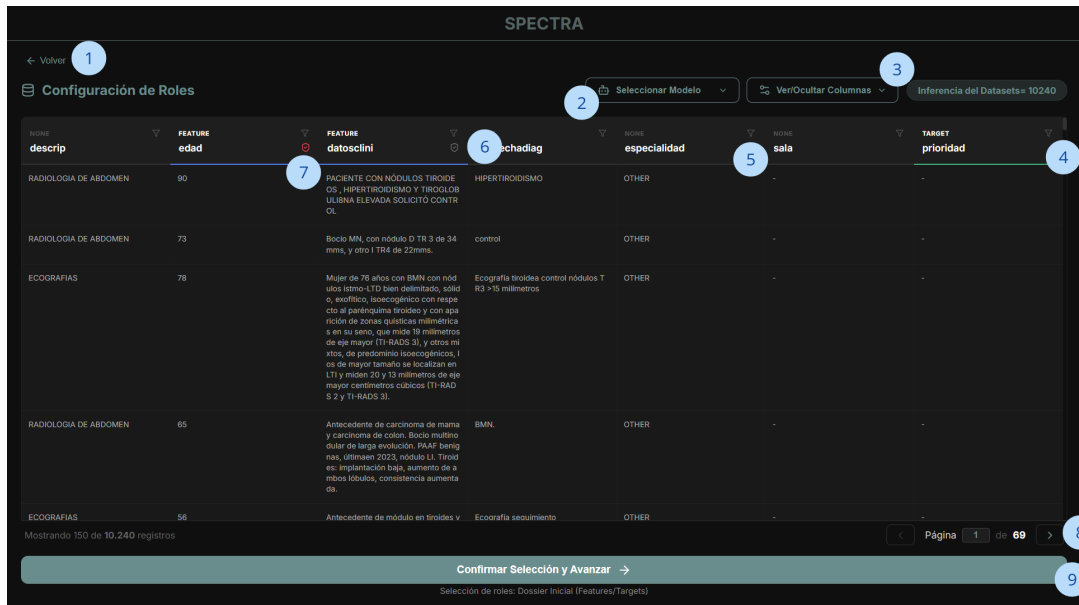


Figura 14. Manual de usuario - Selección de roles y modelo LLM

9.3.3. Descripción de la Interfaz de Análisis y Orquestación

En la Figura 15 se presenta la vista que describe el análisis y la orquestación, donde se visualizan la orquestación, el análisis de variables y las métricas para su selección. Se identifican los siguientes pasos:

1. **Botón de volver.** Permite al usuario interrumpir el proceso y regresar a la pantalla anterior
2. **Botón Seleccionar modelo.** Permite seleccionar el modelo a implementar.
3. **Botón Mostrar columnas.** Permite seleccionar qué columnas se muestran en la tabla.
4. **Botón iniciar generación de script.** Permite enviar los datos para que el modelo LLM seleccionado en el script los genere.
5. **Panel plan Orquestación.** Permite visualizar la estrategia calculada matemáticamente además de la matriz de dependencia de los datos.
6. **Clasificación semántica (nominal o ordinal).** Permite al usuario visualizar y, de ser necesario, editar la información de cada variable categórica cuando el modelo LLM se equivoca.
7. **Mapeo Ordinal.** Permite visualizar y definir, si el modelo LLM se ha equivocado, el orden jerárquico de las variables.
8. **Resumen de ejecución.** Permite visualizar las métricas que el usuario ha seleccionado.
9. **Panel de selección de métricas.** Permite seleccionar las métricas de cada variable para reflejar la confianza del modelo.
10. **Panel de Análisis Técnico.** Permite visualizar la clasificación adicional de las variables que no son categóricas.

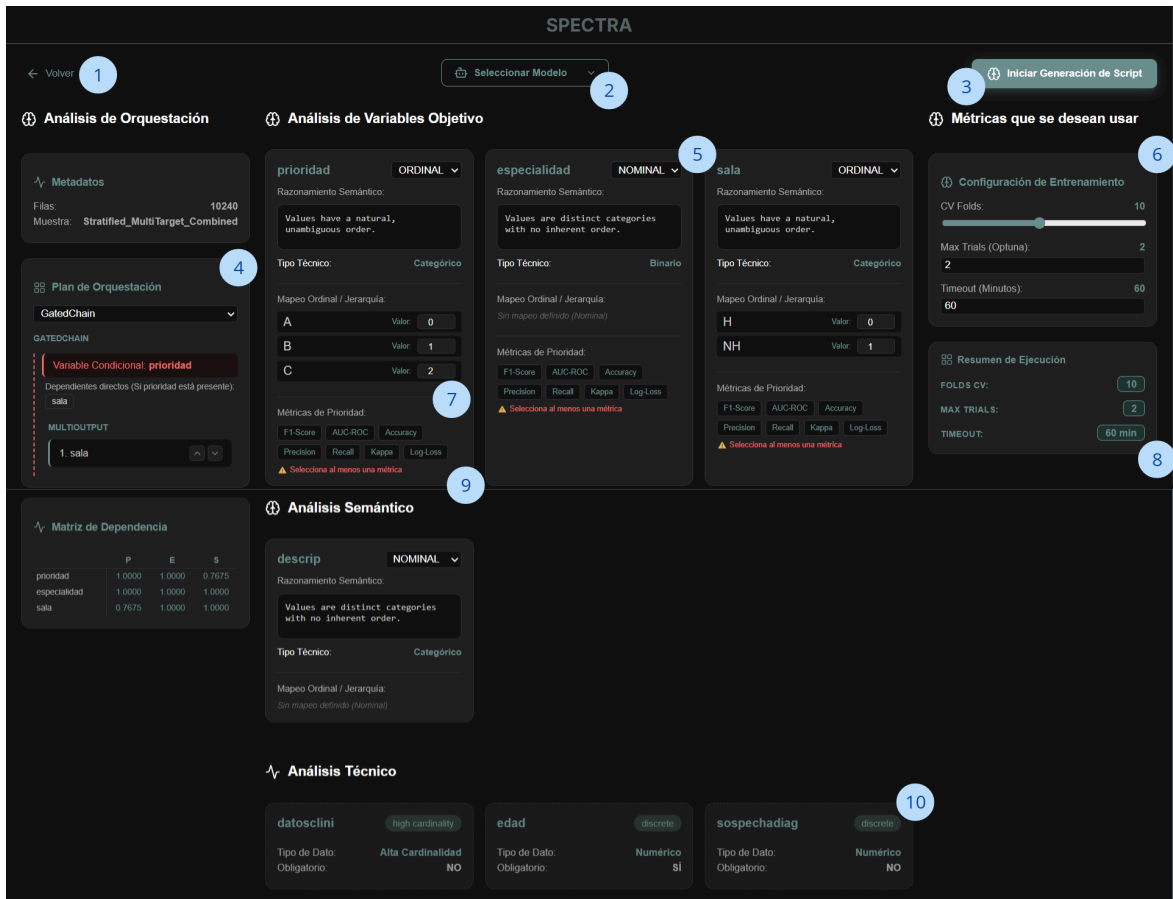


Figura 15. Manual de usuario - Descripción Análisis y Orquestación

9.3.4. Interfaz de Auditoría de Código

En la Figura 16 se presenta la vista de la presentación del script generado, en la que se visualiza de forma editable, con los datos de las métricas seleccionadas a la derecha. Se identifican los siguientes pasos:

1. **Botón de volver.** Permite al usuario interrumpir el proceso y regresar a la pantalla anterior.
2. **Resumen de ejecución.** Permite visualizar las métricas que el usuario ha seleccionado.
3. **Editor de script de entrenamiento.** Permite editar el script si hay alguna métrica incorrecta.
4. **Botón Entrenamiento.** Permite enviar el script final para realizar el entrenamiento.

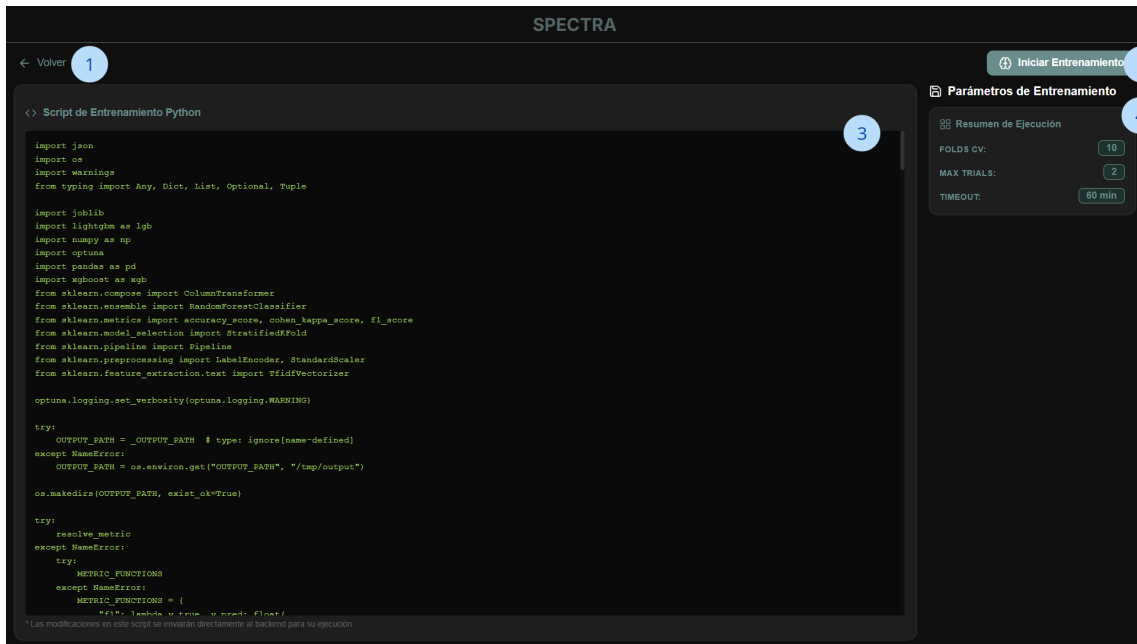


Figura 16. Manual de usuario - Visualización y Edición del Script

9.3.5. Interfaz de Resultado de Entrenamiento

En la Figura 17 se presenta la vista final con los resultados de los modelos de entrenamiento, en la que se visualizan las métricas y la confianza de los modelos. Se identifican los siguientes pasos:

1. **Botón para realizar otro entrenamiento.** Permite al usuario realizar otro entrenamiento regresando a la pantalla inicial de carga del documento.
2. **Panel de Métricas Globales.** Permite visualizar el resumen de la precisión media, los modelos procesados y la estrategia de optimización.
3. **Métricas Detalladas por Objetivo.** Permite visualizar el resumen de entrenamiento de cada variable target.

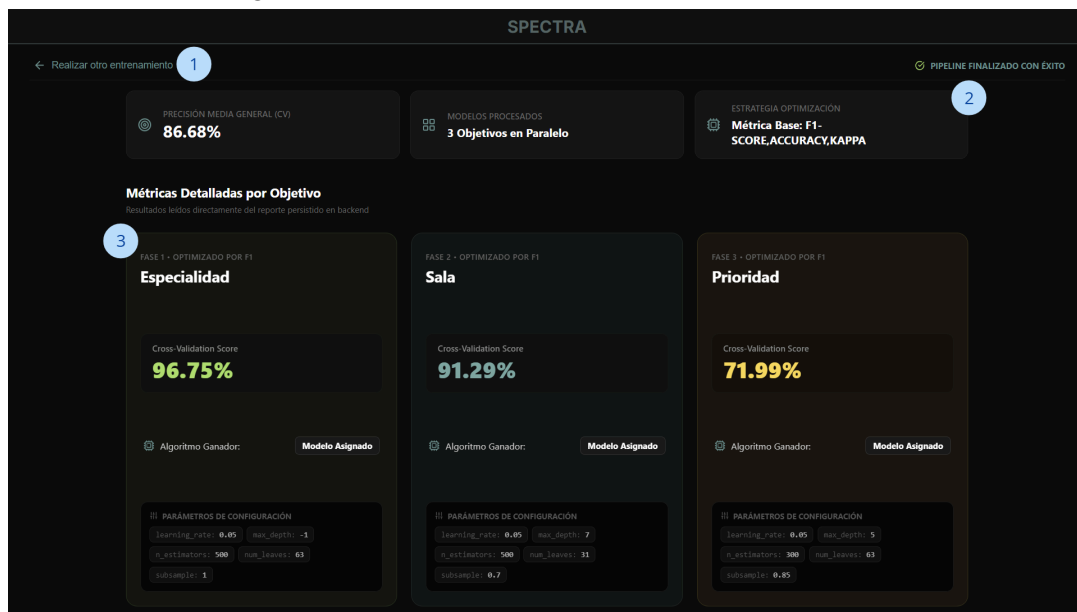


Figura 17. Visualización de las métricas finales

10. Conclusiones y trabajo futuro

En esta sección se presentan las conclusiones derivadas del trabajo desarrollado, se indican las aportaciones realizadas durante el proyecto, así como las líneas de trabajo futuras con las que se puede seguir mejorando el sistema.

10.1. Principales aportaciones

Teniendo en cuenta los objetivos fijados, se puede afirmar que se han realizado las siguientes aportaciones con la realización del trabajo:

- A1.** Desarrollo de un sistema completo de AutoML interactivo y distribuido (SPECTRA), capaz de automatizar la selección de los modelos óptimos para las variables dadas y la generación automática de modelos ya entrenados para su uso posterior.
- A2.** Implementación de un motor de clasificación y perfilado estadístico, llevada a cabo dentro del sistema local, donde se analiza la estructura del dataset para abstraer las propiedades más importantes en un documento de metadatos, además de calcular, mediante análisis estadísticos, la mejor estrategia para el posterior entrenamiento de modelos.
- A3.** Diseño de un orquestador basado en inteligencia artificial mediante la integración de modelos locales y remotos (LLM vía Ollama), capaz de clasificar subtipos de variables categóricas, así como de generar un script funcional que ejecute el entrenamiento de los modelos óptimos seleccionados por sí mismo.
- A4.** Mecanismo de resiliencia y autocorrección de código capaz de interpretar las excepciones que emite la consola y de solicitar parches correctivos al LLM sin intervención humana. La ejecución se realiza en un bucle cerrado, aislado e integrado en un contenedor Docker, hasta que el código funcione correctamente.
- A5.** El entrenamiento de modelos seccionado en dos fases para la optimización de recursos, ya que se realiza un test inicial simple para validar el script, para después realizar el entrenamiento con la configuración que el usuario decida.
- A5.** Implementación de una interfaz web intuitiva que facilita la interacción del cliente con el sistema de entrenamiento.

10.2. Conclusiones

Como conclusión general, la realización del trabajo ha sido muy satisfactoria, ya que se ha conseguido desarrollar un sistema funcional, seguro e inteligentes donde se democratiza el acceso al aprendizaje automático avanzado garantizado así el aislamiento del entorno. Esto ha permitido al alumno aplicar los conocimientos adquiridos durante la carrera en un proyecto centrado en arquitecturas de software novedosas orientadas a la inteligencia artificial.

Desde un punto de vista técnico, el proyecto ha requerido integrar y aplicar conocimientos de diversos campos como (i) la ingeniería de prompts y consumo de LLMs locales y remotas, (ii) la automatización de la infraestructura aislándola con Docker, (iii) análisis estadístico matemático de los datos para la clasificación de los datos y la obtención de arquitectura y dependencias existentes, y (iv) el desarrollo web síncrono y asíncrono. Se han utilizado tecnologías y herramientas como FastAPI, React en el *Back-End* y Vite en el *Front-End* para dotar al sistema de robustez y reactividad para los entornos de producción locales.

La implementación del sistema de resiliencia y autorreparación se ha demostrado ser la fase de mayor complejidad técnica de la aplicación. Capturar el flujo de errores dentro de un contenedor Docker en tiempo real, procesar las excepciones y forzar al LLM a corregir su propio código bajo un umbral

estricto de intentos representa un enfoque avanzado de ingeniería de software orientada a mejorar la fiabilidad en los modelos generativos.

10.3. *Vías de trabajo futuro*

Los resultados obtenidos sugieren la posibilidad de delegar la toma de decisiones sobre el entrenamiento de modelos a un LLM local, sólo si este está guiado por perfiles matemáticos estructurados. Al combinar el razonamiento conceptual con los sistemas estadísticos calculados en tiempo de ejecución, SPECTRA ofrece un equilibrio entre la automatización inteligente, la seguridad técnica y el control humano.

Para finalizar el proyecto, se presenta una serie de ampliaciones futuras para el sistema SPECTRA, que incorporan funcionalidades no incluidas debido a la limitación temporal de este Trabajo de Fin de Grado, pero que consolidarán el sistema como un producto de grado empresarial.

Una mejora de gran utilidad sería la implementación de capacidades de MLOps y de despliegue automático (Model Serving). Actualmente, el sistema serializa los modelos óptimos en archivos .joblib del manager local. Un trabajo futuro consistiría en desarrollar un módulo que exponga automáticamente estos modelos recién entrenados como microservicios independientes (APIs REST) o como contenedores Docker, permitiendo a otras aplicaciones consumir las predicciones en tiempo real sin intervención manual.

Con respecto al *Sandbox* de AutoML, se podría ampliar el catálogo de algoritmos incorporando modelos de Deep Learning. Aunque las topologías basadas en árboles (*XGBoost*, *LightGBM*) han demostrado ser una excelente opción para este caso de datos, la integración de redes neuronales profundas (mediante PyTorch o TensorFlow) en la fase de optimización con Optuna permitiría abordar conjuntos de datos clínicos u otros de mayor dimensionalidad.

Por último, sería interesante desarrollar un módulo de monitorización de Data Drift y de Reentrenamiento Continuo. En entornos reales, la distribución de las variables puede cambiar a medida que pasa el tiempo, además tener más datos de entrenamiento para ser más preciso. El sistema podría evaluar de forma automatizada la calidad de las predicciones en producción y, al detectar una degradación, disparar un nuevo ciclo de entrenamiento en el Laboratorio de Modelos para autoajustarse a los nuevos datos y mantener los niveles de confianza.

11. Referencias

- [D1] Zöller, M.-A., & Huber, M. F. (2021). Benchmark and Survey of Automated Machine Learning Frameworks. *Journal of Artificial Intelligence Research*, 70, 409–472. DOI: <https://doi.org/10.1613/jair.1.11854>
- [D2] Karmaker, S. et al. (2021). AutoML to Date and Beyond: Challenges and Opportunities. *ACM Computing Surveys*, 54(8), 1–36. DOI: <https://doi.org/10.1145/3470918>
- [D3] Feurer, M. et al. (2015). Efficient and Robust Automated Machine Learning. *Advances in Neural Information Processing Systems*, 28 DOI: 10.5555/2969442.2969547
- [D4] Erickson, N. et al. (2020). AutoGluon-Tabular: Robust and Accurate AutoML for Structured Data. *arXiv preprint arXiv:2003.06505*. DOI: 10.48550/arXiv.2003.06505
- [D5] LeDell, E., & Poirier, S. (2020). H2O AutoML: Scalable Automatic Machine Learning. *Proceedings of the AutoML Workshop at ICML 2020*. DOI: 10.1088/1749-4699/8/1/014008
- [D6] Olson, R. S., & Moore, J. H. (2016). TPOT: A Tree-based Pipeline Optimisation Tool for Automating Machine Learning. *Proceedings of the Workshop on Automatic Machine Learning*, PMLR 64, 66–74. DOI: 10.1007/978-3-030-05318-5_8
- [D7] Xin, D. et al. (2023). AutoML in The Wild: Obstacles, Workarounds, and Expectations. *En Proceedings of the CHI Conference on Human Factors in Computing Systems*, ACM DOI: 10.1145/3544548.3581082
- [D8] Tornede, T. et al. (2023). AutoML in the Age of Large Language Models: Current Challenges, Future Opportunities and Risks. *arXiv preprint arXiv:2306.08107*. DOI: 10.48550/arXiv.2306.08107
- [D9] Luo, Z. et al. (2024). Large Language Models Synergise with Automated Machine Learning. *arXiv preprint arXiv:2405.03727* DOI: 10.48550/arXiv.2405.03727
- [D10] Zheng, A., & Casari, A. (2018). *Feature Engineering for Machine Learning: Principles and Techniques for Building Robust Models*. O'Reilly Media.
- [D11] Xu, S. et al. (2019). A Survey on Multi-output Learning. *IEEE Transactions on Neural Networks and Learning Systems*, 31(7), 2409–2429. DOI: 10.48550/arXiv.1901.00248
- [D12] Chen, J. et al. (2023). Deep Generative Imputation Model for Missing Not At Random Data. *arXiv preprint arXiv:2308.08158*. DOI: 10.1145/3583780.3614835
- [D13] Concepto. (s.f.). Diagrama de Gantt: qué es, cómo hacer uno y ejemplos. *Encyklopedie Concepto*. <https://concepto.de/diagrama-de-gantt/>
- [D14] Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4), 427-437.
- [D15] Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1), 37-46.

Anexo A. Diagramas de Gantt

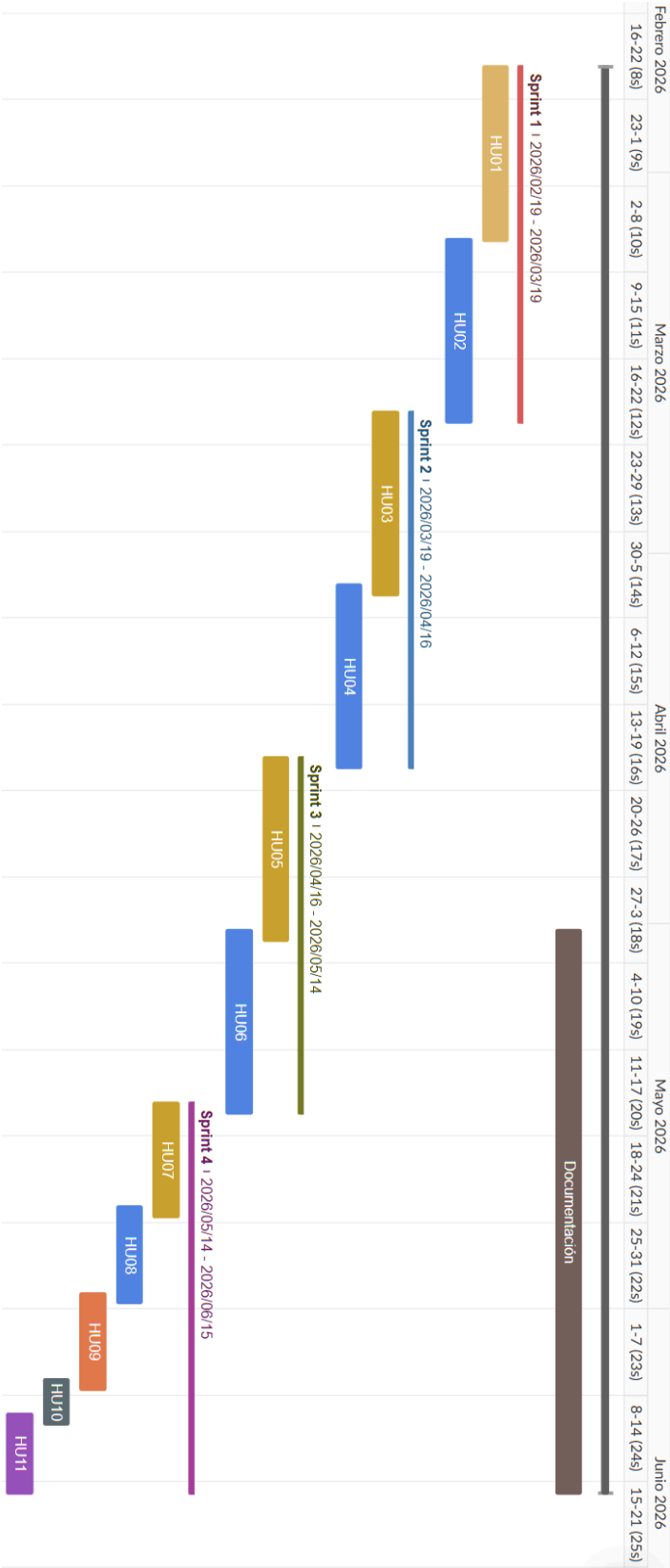


Figura 18. Diagrama de Gantt - Resumen de la Estimación

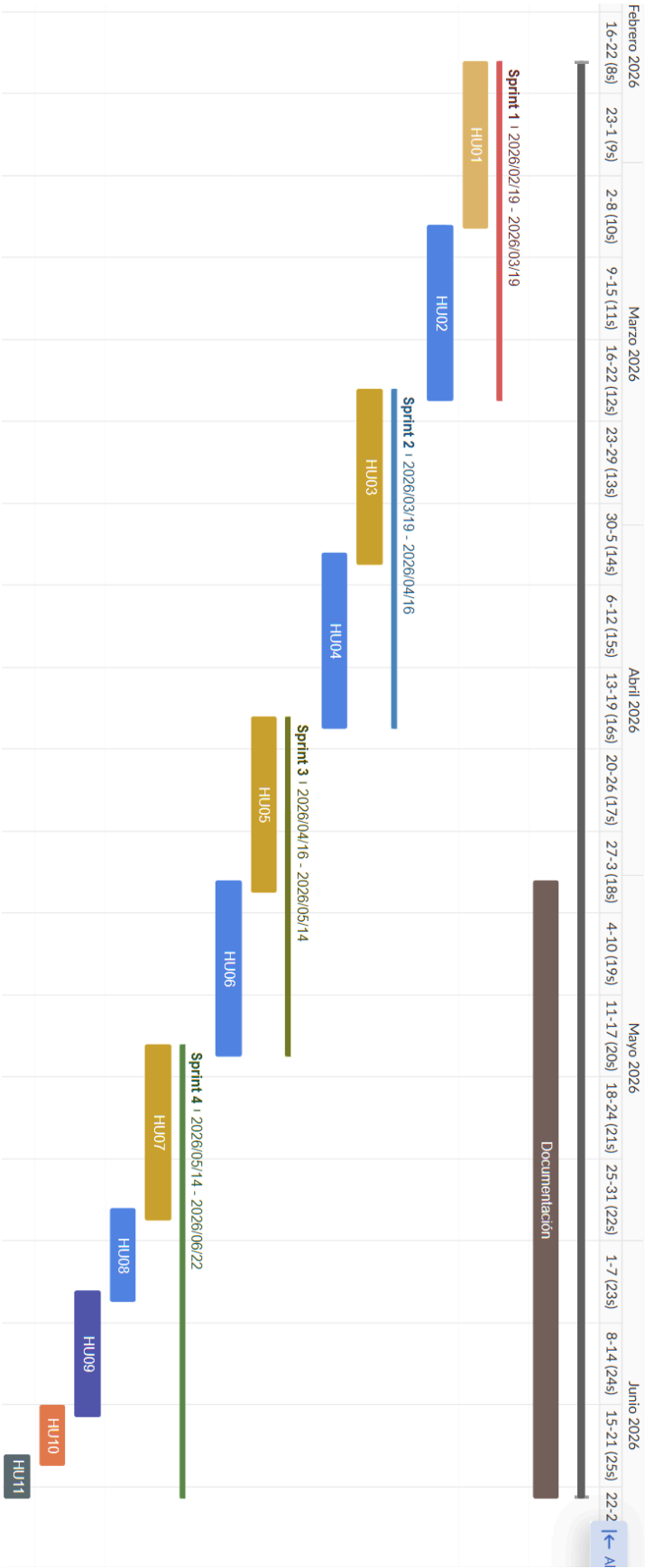


Figura 19. Diagrama de Gantt - Resumen de la Ejecución

Anexo B. Diagramas

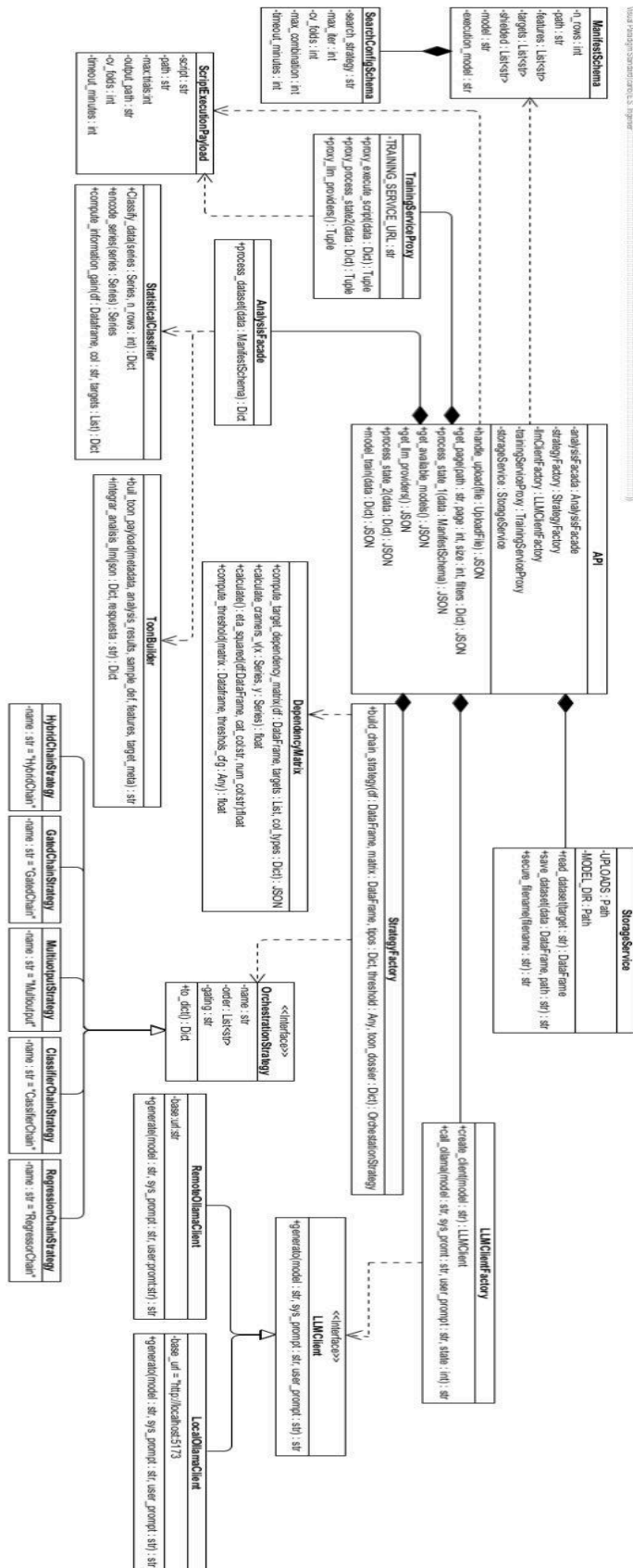


Figura 20. Diagrama general de clases del sistema

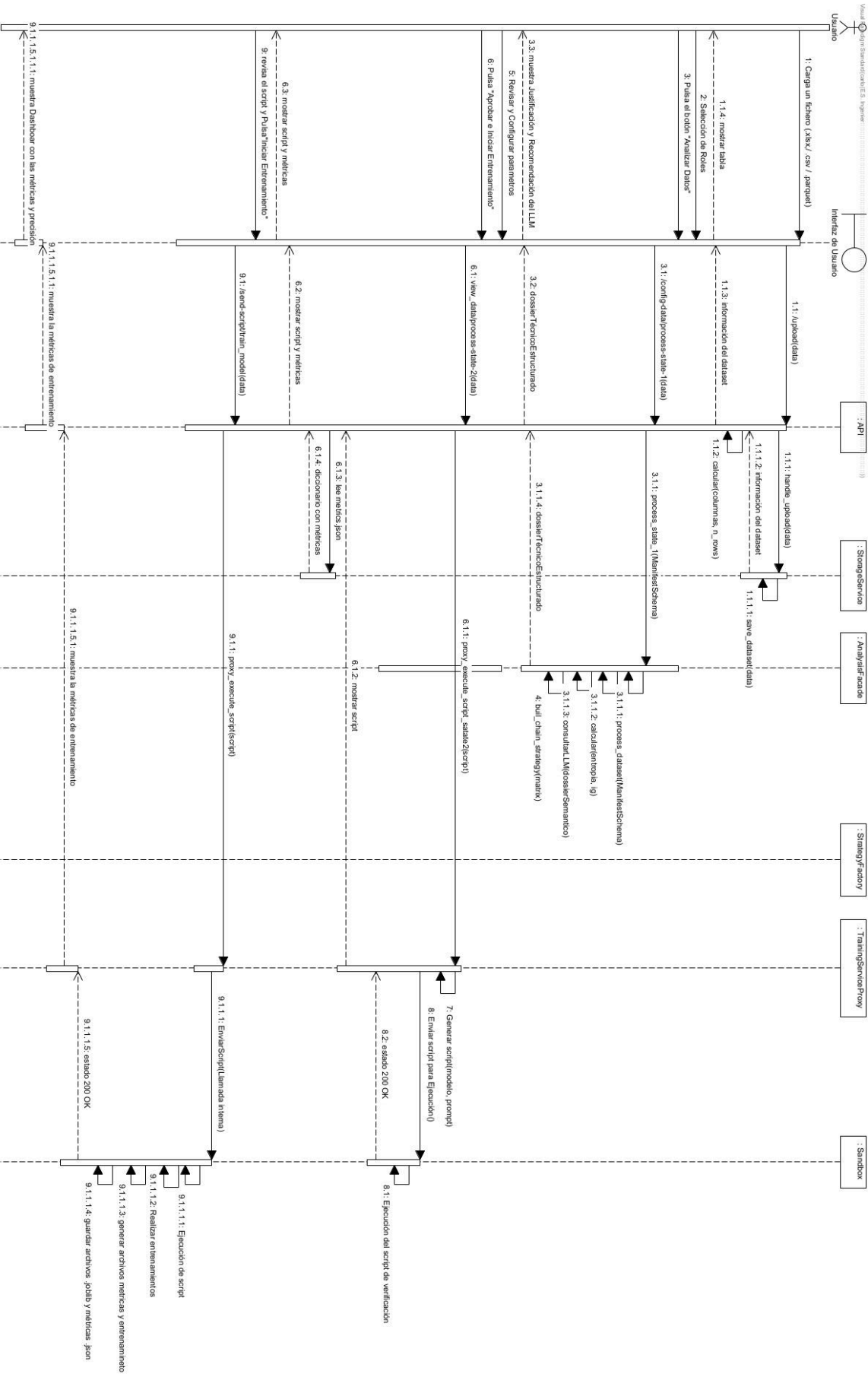


Figura 21. Diagrama de secuencia general

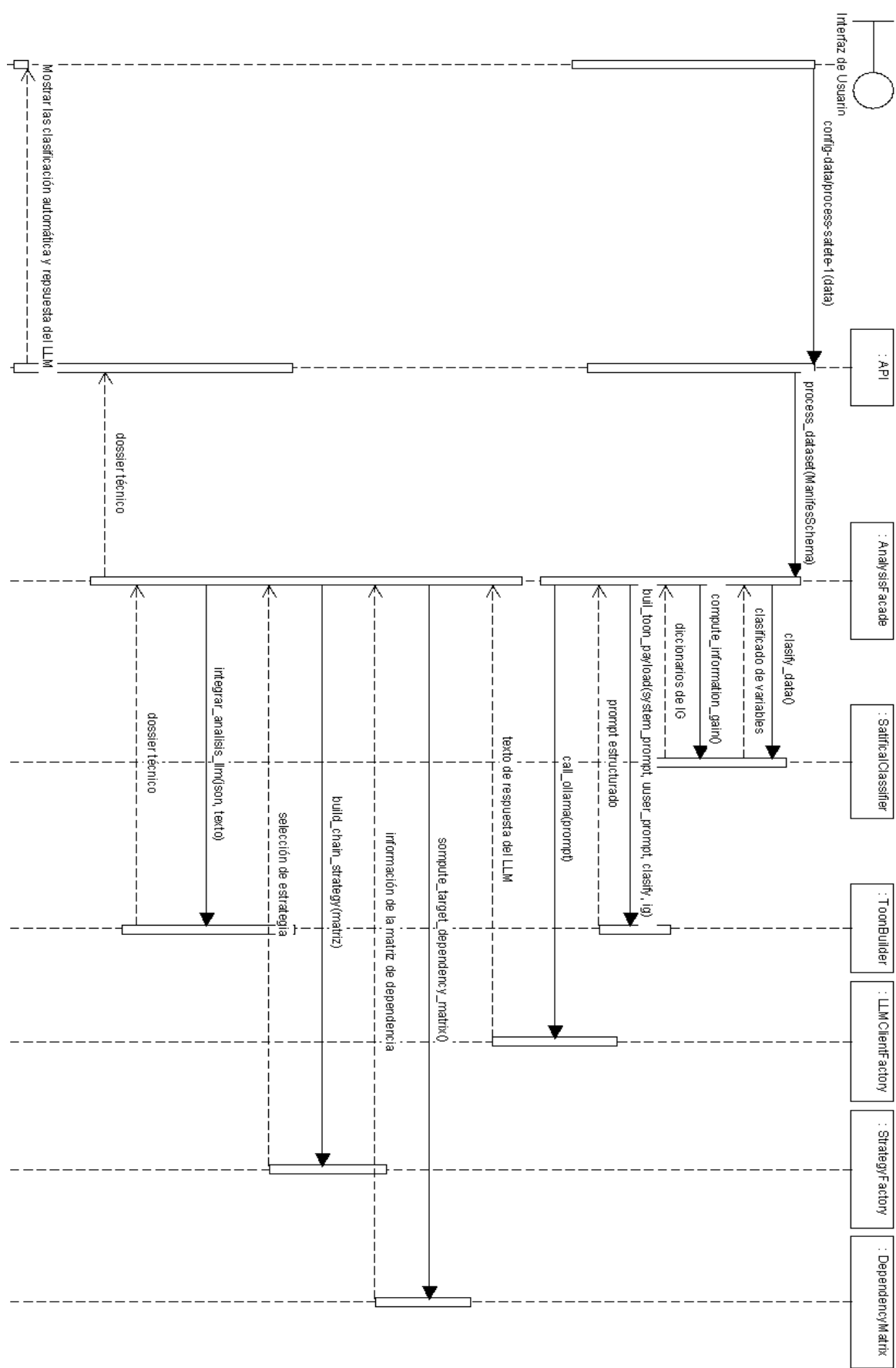


Figura 22. Diagrama de secuencia de Análisis y Orquestación de Datos

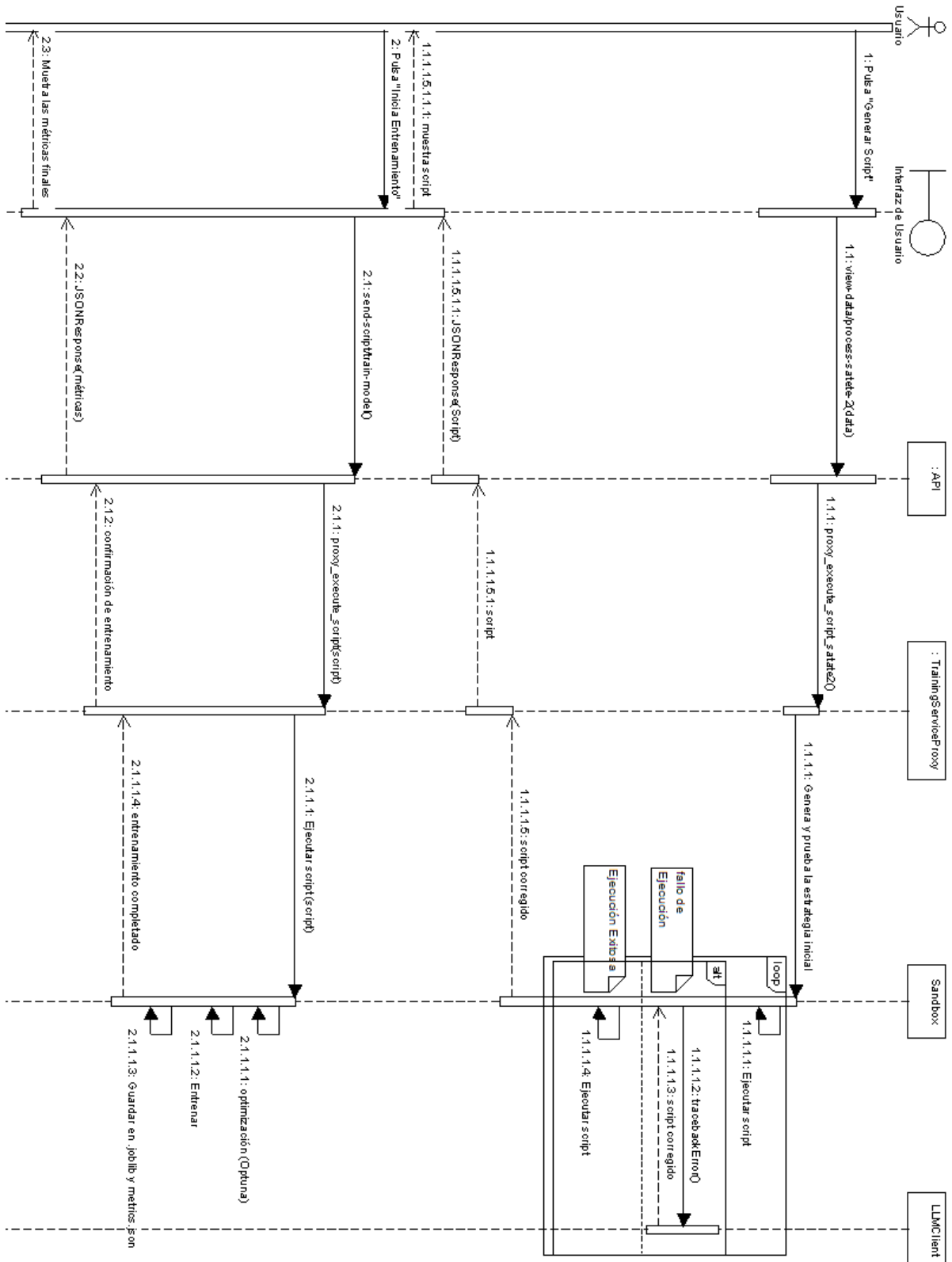


Figura 23. Diagrama de secuencia de Laboratorio AutoML

Anexo C. Pruebas Realizadas

Tabla 7. Conjunto de pruebas de caja negra realizadas

HU01 Carga, validación de integridad de datos		
Prueba	Resultado esperado	Resultado Obtenido
Se carga un archivo CSV, Excel o Parquet estándar bien estructurado a través de la interfaz.	El sistema parsea el archivo, extrae el número de filas y avanza a la siguiente fase sin problemas.	El sistema parsea el archivo, extrae el número de filas y avanza a la siguiente fase sin problemas.
Se carga un dataset legítimo que contiene celdas completamente vacías o cadenas como "NA" o "NULL".	El archivo se carga con éxito. El sistema mapea correctamente esas celdas como valores nulos matemáticos (NaN) para tratarlos posteriormente.	El archivo se carga con éxito. El sistema mapea correctamente esas celdas como valores nulos matemáticos (NaN) para tratarlos posteriormente.
Se intenta cargar un archivo corrupto o en un formato incompatible (por ejemplo, un PDF).	Error	Error
HU02 Interfaz de selección de roles y declaración de variables blindadas		
Prueba	Resultado esperado	Resultado Obtenido
En la interfaz se seleccionan múltiples columnas como targets y el resto como features y se selecciona el modelo LLM.	El sistema registra correctamente el diccionario de roles y lo inyecta en el manifiesto principal.	El sistema registra correctamente el diccionario de roles y lo inyecta en el manifiesto principal.
El usuario marca una o varias features con el flag de la variable blindada.	El sistema registra esta orden en la lista shielded del manifiesto, lo que obliga al LLM a incluirla en el modelo por motivos de negocio, independientemente de su estadística.	El sistema registra esta orden en la lista shielded del manifiesto y obliga a su inclusión posterior.
El usuario intenta avanzar a la fase de entrenamiento sin haber seleccionado ningún target.	Error	Error
El usuario selecciona uno o varios targets, pero no ninguna feature.	Error	Error

El usuario selecciona uno o varios targets y una o varias features, pero no ningún modelo de LLM.

Error

Error

HU03 Motor estadístico local

Prueba	Resultado esperado	Resultado Obtenido
Se carga un dataset con variables binarias, categóricas, numéricas y de alta cardinalidad.	El sistema calcula el ratio de unicidad de cada columna, asigna automáticamente el tipo (1, 2, 3 o 4) y lo documenta en el TOON.	El sistema calcula el ratio de unicidad de cada columna, asigna automáticamente el tipo (1, 2, 3 o 4) y lo documenta en el TOON.
Se procesa un dataset que contiene valores NaN y distribuciones numéricas muy asimétricas.	El sistema aplica imputaciones temporales (moda/mediana) y transformaciones matemáticas (log1p/yeo-johnson) a una copia de los datos antes de calcular la entropía (Information Gain).	El sistema aplica imputaciones temporales (moda/mediana) y transformaciones matemáticas (log1p/yeo-johnson) a una copia de los datos antes de calcular la entropía (Information Gain).
El dataset tiene dos targets ("sala" y "prioridad") y dimensiones suficientes para el muestreo.	El sistema construye una columna temporal ("sala_prioridad"), extrae exactamente 100 filas representativas de forma estratificada y purga la columna combinada resultante.	El sistema construye una columna temporal ("sala_prioridad"), extrae exactamente 100 filas representativas de forma estratificada y purga la columna combinada resultante.

HU04 Clasificación semántica mediante LLM con cálculo dinámico

Prueba	Resultado esperado	Resultado Obtenido
Se inyecta el TOON dossier con la muestra de 100 filas en el modelo local (Ollama).	El sistema evalúa la arquitectura del modelo y la VRAM disponible para calcular un techo dinámico de contexto (num_ctx) y evitar saturar la memoria.	El sistema evalúa la arquitectura del modelo y la VRAM disponible para calcular un techo dinámico de contexto (num_ctx) y evitar saturar la memoria.
El LLM evalúa una variable categórica que no tiene un orden matemático inherente (ej. departamento, sala).	El LLM la clasifica correctamente como NOMINAL, justificando su razonamiento semántico en el dossier.	El LLM la clasifica correctamente como NOMINAL, justificando su razonamiento semántico en el dossier.
El LLM evalúa una variable categórica que representa una jerarquía o progreso lógico (ej. nivel_cargo o prioridad).	El LLM la clasifica como ORDINAL y genera además un diccionario de mapeo numérico respetando la estructura ordinal (ej. Junior: 0, Senior: 1).	El LLM la clasifica como ORDINAL y genera además un diccionario de mapeo numérico respetando la estructura ordinal (ej. Junior: 0, Senior: 1).

HU05 Evaluación orquestadora multiobjetivo de las dependencias de los targets

Prueba	Resultado esperado	Resultado Obtenido
Se configura un dataset con dos targets de tipo 1 o 2 (p. ej., especialidad y sala).	El sistema detecta la combinación categórica, genera la tabla cruzada (crosstab) y calcula la correlación χ^2 + Cramér's V, devolviendo un índice preciso en el rango [0, 1].	El sistema detecta la combinación categórica, genera la tabla cruzada (crosstab) y calcula la correlación χ^2 + Cramér's V, devolviendo un índice preciso en el rango [0, 1].
Se evalúan dos targets continuos o de alta cardinalidad (Tipo 3 o 4) (p. ej., precio y días_ingreso).	El sistema detecta tipos numéricos y aplica la correlación de rangos de Spearman, devolviendo el valor absoluto del coeficiente `	El sistema detecta tipos numéricos y aplica la correlación de rangos de Spearman, devolviendo el valor absoluto del coeficiente `
Se cruza un target categórico contra uno numérico en la orquestación.	El sistema agrupa la variable numérica por las clases de la categórica, aplica el test de Kruskal-Wallis y deriva el tamaño del efecto mediante Eta Cuadrado (η^2).	El sistema agrupa la variable numérica por las clases de la categórica, aplica el test de Kruskal-Wallis y deriva el tamaño del efecto mediante Eta Cuadrado (η^2).
Se envía la matriz de dependencias completa al evaluador dinámico.	El algoritmo extrae todos los scores, ignorando la diagonal principal (donde el score es artificialmente 1.0), y define el umbral sumando la mediana a la desviación típica de las dependencias reales.	El algoritmo extrae todos los scores, ignorando la diagonal principal (donde el score es artificialmente 1.0), y define el umbral sumando la mediana a la desviación típica de las dependencias reales.
El usuario llega a esta fase de orquestación pero el manifiesto indica que solo hay 1 target seleccionado.	El sistema detecta $\text{len}(\text{targets}) == 1$, omite la creación de la matriz simétrica de dependencias al no tener pares con los que comparar, y pasa directamente a la selección del pipeline sin lanzar error.	El sistema detecta $\text{len}(\text{targets}) == 1$, omite la creación de la matriz simétrica de dependencias al no tener pares con los que comparar, y pasa directamente a la selección del pipeline sin lanzar error.

HU06 Módulo de detección de ausencias estructurales y decisión de estrategias

Prueba	Resultado esperado	Resultado Obtenido
Se detecta que un Target tiene valores nulos (NaN) única y exclusivamente cuando otro Target tiene un valor específico (MNAR - Ausencia Estructural).	El sistema prioriza esta condición, anula las correlaciones artificiales e impone de inmediato la estrategia maestra GatedChain.	El sistema prioriza esta condición, anula las correlaciones artificiales e impone de inmediato la estrategia maestra GatedChain.
Los targets superan el umbral de dependencia y están	El sistema recorre el árbol de decisiones y selecciona	El sistema recorre el árbol de decisiones y selecciona

mezclados (categóricos y numéricos).	HybridChain, ordenando sus predicciones de forma descendente según su influencia.	HybridChain, ordenando sus predicciones de forma descendente según su influencia.
Los targets superan el umbral y son todos categóricos: uno ordinal y otro nominal.	El sistema selecciona ClassifierChain e instruye al orquestador a utilizar un modelo de clasificación estándar para el Nominal y métricas/modelos ordinales para el otro.	El sistema selecciona ClassifierChain e instruye al orquestador a utilizar un modelo de clasificación estándar para el Nominal y métricas/modelos ordinales para el otro.
El valor max_dep calculado en toda la matriz no alcanza el umbral definido.	La función asume una baja covarianza, ignora el cálculo de la influencia cruzada y adopta la estrategia plana MultiOutput.	La función asume una baja covarianza, ignora el cálculo de la influencia cruzada y adopta la estrategia plana MultiOutput.
La matriz detecta una alta dependencia y se comprueba que todos los targets pertenecen a los tipos 3 o 4 (Numéricos/Continuos).	La validación detecta homogeneidad numérica (todos_num = True) y deriva correctamente a la arquitectura RegressorChain.	La validación detecta homogeneidad numérica (todos_num = True) y deriva correctamente a la arquitectura RegressorChain.

HU07 Generación automática del script de entrenamiento y prueba rápida

Prueba	Resultado esperado	Resultado Obtenido
El usuario selecciona métricas válidas, elige un modelo de LLM de la lista y presiona el botón de generación.	El sistema interactúa con el modelo LLM correctamente y pinta en pantalla el código fuente Python generado listo para el entorno <i>Sandbox</i> .	El sistema interactúa con el modelo LLM correctamente y pinta en pantalla el código fuente Python generado listo para el entorno <i>Sandbox</i> .
El usuario intenta avanzar y generar el script dejando el desplegable de "Modelo LLM" vacío o sin seleccionar.	Error	Error
A pesar de las instrucciones sistémicas, el LLM alucina y responde con "¡Claro! Aquí tienes el script..." antes del bloque de código.	El módulo de limpieza extrae el código fuente puro ignorando el texto conversacional, asegurando un script ejecutable.	El módulo de limpieza extrae el código fuente puro ignorando el texto conversacional, asegurando un script ejecutable.

HU08 Aislamiento del entorno de ejecución mediante *Sandbox*

Prueba	Resultado esperado	Resultado Obtenido
El sistema valida que Docker esté instalado y en ejecución en la máquina host, y que el modo de ejecución esté en auto.	Docker está disponible; el sistema crea un contenedor de python:3.11-slim, inyecta el script y monta el dataset original en modo estricto de	Docker está disponible; el sistema crea un contenedor de python:3.11-slim, inyecta el script y monta el dataset original en modo estricto de

	solo lectura (:ro).	solo lectura (:ro).
El servicio de Docker está apagado o el usuario fuerza manualmente el modo a subprocess.	El sistema realiza un fallback a la ejecución local mediante un subprocesso de Python protegido por un timeout predefinido.	El sistema realiza un fallback a la ejecución local mediante un subprocesso de Python protegido por un timeout predefinido.

HU09 Implementación del bucle cerrado de autocorrección del código

Prueba	Resultado esperado	Resultado Obtenido
El script de entrenamiento generado aborta su primera ejecución al lanzar un error en la consola, de tipo SyntaxError o IndentationError.	El bucle detecta el error, lo cataloga como sintáctico, y hace un prompt exigiendo un arreglo de una sola línea al LLM sin alterar los modelos base.	El bucle detecta el error, lo cataloga como sintáctico, y hace un prompt exigiendo un arreglo de una sola línea al LLM sin alterar los modelos base.
El script falla en su segundo intento y arroja un error de compatibilidad de tensores/dimensiones (shape mismatch).	El bucle de autorreparación lo clasifica como un error estructural y permite al LLM reescribir bloques más grandes del pipeline de transformación.	El bucle de autorreparación lo clasifica como un error estructural y permite al LLM reescribir bloques más grandes del pipeline de transformación.
El entorno <i>Sandbox</i> colapsa porque la máquina se queda sin recursos, lo que provoca un MemoryError o un error de disco lleno (errno 28).	El sistema lanza el trigger UNRECOVERABLE_ERROR y aborta inmediatamente la ejecución, sin reintentos inútiles.	El sistema lanza el trigger UNRECOVERABLE_ERROR y aborta inmediatamente la ejecución, sin reintentos inútiles.
Se produce un error lógico extraordinariamente difícil de reparar. El LLM de reparación falla repetidamente (p. ej., genera el mismo error).	El bucle alcanza el intento número 3 sin poder compilar el script; devuelve un dict de fracaso y evita el bloqueo perpetuo de la máquina.	El bucle alcanza el intento número 3 sin poder compilar el script; devuelve un dict de fracaso y evita el bloqueo perpetuo de la máquina.

HU10 Virtualización del código e interacción para entrenamiento pesado

Prueba	Resultado esperado	Resultado Obtenido
Tras la respuesta del modelo orquestador (HU07), la interfaz transiciona a la vista final previa al entrenamiento.	El editor de texto interactivo muestra el código fuente completo en Python, con las métricas seleccionadas a su derecha, listo para ser revisado.	El editor de texto interactivo muestra el código fuente completo en Python, con las métricas seleccionadas a su derecha, listo para ser revisado.
El usuario valida que el código es correcto (con o sin ediciones manuales) y presiona el botón de inicio del entrenamiento.	El botón de entrenamiento dispara asincrónicamente el <i>Back-End</i> , activando la ejecución virtualizada sin colgar ni bloquear la página web.	El botón de entrenamiento dispara asincrónicamente el <i>Back-End</i> , activando la ejecución virtualizada sin colgar o bloquear la página web.
El usuario inspecciona el	El sistema almacena la edición	El sistema almacena la edición

código, decide que prefiere usar un cv-fold menor en lugar de la métrica seleccionada anteriormente, y edita directamente la línea en el código.	humana, sobrescribe el script en memoria e inyecta la versión corregida por el usuario en el <i>Sandbox</i> .	humana, sobrescribe el script en memoria e inyecta la versión corregida por el usuario en el <i>Sandbox</i> .
---	--	--